UIUC CS 374

Introduction to Algorithms and Models of Computation

Anirudh Konidala

August 2026 – December 2026 (Fall 2026)

Contents

1 Lecture 01 (Aug 25) - Introduction to CS 374 6
1.1 Alphabets 6
1.2 Strings 6
1.2.1 Example 6
1.3 Functions 6
1.3.1 Example 6
1.4 Concatenation 6
1.4.1 Example 7
1.5 Induction 7
1.5.1 Example 1 7
1.5.2 Example 2 7
1.5.3 Example 3 8
2 Lecture 10 (Sep 29) - Recursion: Tower of Hanoi and Mergesort 9
2.1 The RealRAM Model 9
2.2 Describing Algorithms 9
2.3 Reductions and Recursion 10
2.3.1 Example: sum of an array 10
2.3.2 The recursion fairy 10
2.4 Tower of Hanoi 10
2.4.1 The recursive idea 11
2.4.2 Algorithm 11
2.4.3 Counting moves 11
2.4.4 Proof by induction 12
2.5 Mergesort 12
2.5.1 Merging 12
2.5.2 Recurrence 13
2.6 Recursion Trees 13
2.6.1 Mergesort 14
2.6.2 The three common shapes 14
3 Lecture 11 (Oct 1) - Divide and Conquer: Quicksort and Linear Time Selection 16
3.1 Divide and Conquer 16
3.2 Quicksort 16
3.2.1 Rank 16
3.2.2 Partition 17
3.2.3 Worst case 17
3.2.4 Worst-case analysis 18
3.2.5 Good pivots 18
3.3 Selection 18

3.3.1 Sort first 19
3.3.2 Recursive idea 19
3.3.3 The selection problem 19
3.4 Quickselect 19
3.4.1 Running time 20
3.5 Median of Medians Selection 20
3.5.1 Analysis 21
3.5.2 Why blocks of 5? 22
3.5.3 Takeaways 22
3.6 Preview: Fast Multiplication 22
4 Lecture 12 (Oct 6) - Divide and Conquer: Multiplication; Backtracking: Text Segmentation 23
4.1 Fast Multiplication 23
4.1.1 Cost model 23
4.1.2 Split into halves 23
4.1.3 Analysis 24
4.2 Karatsuba's Algorithm 24
4.2.1 Analysis 25
4.2.2 Later history 26
4.2.3 Recursion tree shapes, all three seen 26
4.3 Backtracking 26
4.4 Text Segmentation 27
4.4.1 Decisions 27
4.4.2 The recursive subproblem 27
4.4.3 Running time 28
4.4.4 Pass indices, not arrays 28
4.4.5 As a recurrence 28
4.5 Summary: the Backtracking Recipe 28
5 Lecture 13 (Oct 8) - Dynamic Programming: Fibonacci Numbers, Faster Text Segmentation 30
5.1 Fibonacci Numbers 30
5.1.1 Recursive algorithm 30
5.1.2 Why so slow? 30
5.2 Memoization 31
5.2.1 Analysis 31
5.3 Iterative Fibonacci 32
5.4 Dynamic Programming 32
5.5 Faster Text Segmentation 33
5.5.1 Part 1: Formulate recursively 33
5.5.2 Part 2: Build solutions bottom-up 34

5.6 Summary: the Dynamic Programming Recipe 35
6 Lecture 15 (Oct 15) - Dynamic Programming on Trees: Woodcutting, Maximum Independent Set 36
6.1 Woodcutting 36
6.1.1 The problem 36
6.1.2 Part 1: Formulate recursively 36
6.1.3 Part 2: Build solutions bottom-up 37
6.2 Maximum Independent Set in Trees 39
6.2.1 The problem 39
6.2.2 Part 1: Formulate recursively 39
6.2.3 Part 2: Build solutions bottom-up 40
6.2.4 A cleaner version: two subproblems 40
6.3 Summary 42
7 Lecture 16 (Oct 20) - Graphs: Definitions, Representations, Search, Reductions 43
7.1 What Is a Graph? 43
7.1.1 Vertices can be anything 43
7.2 Representing a Graph 44
7.2.1 Adjacency matrix 44
7.2.2 Adjacency list 44
7.3 Reachability and Whatever-First Search 44
7.3.1 The idea 45
7.3.2 Running time 46
7.4 Reductions 46
7.4.1 Example: useless DFA states 46
8 Lecture 17 (Oct 22) - Graph Layering, Whatever-First Search Variants, Depth-First Search 48
8.1 Graph Layering 48
8.1.1 The problem 48
8.1.2 Don't modify the search, build a new graph 48
8.2 Whatever-First Search, Revisited 49
8.2.1 Parent pointers 49
8.2.2 The bag matters now 50
8.3 Depth-First Search 51
8.3.1 The recursive version 51
8.3.2 Three facts about DFSAll 51
8.3.3 Clock: start and finish times 51
8.4 Finding a Cycle 53
9 Lecture 18 (Oct 27) - Cycle Detection, Topological Sort, Shortest Paths, Dijkstra's Algorithm 54
9.1 Recap: DFS with a Clock 54

9.2 Cycle Detection 54
9.2.1 Spotting back edges with finish times 54
9.3 Topological Sort 55
9.4 Single-Source Shortest Paths 56
9.4.1 The shortest-path tree 56
9.5 Ford's Generic Algorithm 56
9.6 Dijkstra's Algorithm 57
9.6.1 Running time 58
A Final Review: Patterns and Connections 60
A.1 Which Technique? 60
A.2 Algorithms at a Glance 61
A.3 Solving Recurrences: Recursion Trees 63
A.4 Recipes 64
A.4.1 Induction proof (Lec 01) 64
A.4.2 Describing an algorithm (Lec 10) 64
A.4.3 Backtracking (Lec 12) 64
A.4.4 Dynamic programming (Lec 13) 65
A.4.5 DP on a tree (Lec 15) 65
A.4.6 Graph reduction (Lec 16) 65
A.4.7 Graph layering (Lec 17) 65
A.4.8 Shortest paths (Lec 18) 66
A.4.9 Graph representations (Lec 16) 66
A.5 Connections 66
A.6 Common Ways to Lose Points 67
B Labs 69
B.1 Lab 1A: String Induction 69
B.2 Lab 1B: Regular Expressions 72
C Problem Sets 73
C.1 Problem Set 1 73
C.1.1 Problem 1 73

1 Lecture 01 (Aug 25) - Introduction to CS 374

1.1 Alphabets

Let Σ be an **alphabet**, an arbitrary **finite** set of **symbols** or **characters**

Examples include:

- $\Sigma = \{A, B, ..., Z\}$
- $\Sigma = \{0, 1\}$

1.2 Strings

A **string** or **word** is a **finite** sequence of symbols from an alphabet

Formally speaking, w is a string over Σ if it is one of

- the **empty string**
- an ordered pair (a, x), where $a \in \Sigma$ and x is a string over Σ

1.2.1 Example

$$\begin{aligned} STRING &= (S, (TRING)) \\ &= (S, (T, (RING))) \\ &= ... \\ &= (S, (T, (R, (I, (N, (G, \varepsilon)))))) \end{aligned}$$

1.3 Functions

Functions are defined recursively, where length $|w|$ represents the number of symbols in w

$$|w| := \begin{cases} 0 & \text{if } \ w = \varepsilon \\ 1 + |x| & \text{if } \ w = ax \end{cases}$$

1.3.1 Example

$$\begin{aligned} |HAT| &= 1 + |AT| \\ &= 1 + (1 + |T|) \\ &= 1 + (1 + (1 + |\varepsilon|)) \\ &= \boxed{3} \end{aligned}$$

1.4 Concatenation

Let **concatenation**, $w \bullet z$, of strings w and z be represented as

$$w \bullet z := \begin{cases} z & \text{if } \ w = \varepsilon \\ (a, x \bullet z) & \text{if } \ w = ax \end{cases}$$

1.4.1 Example

$$\begin{aligned} NOW \bullet HERE &= N \cdot (OW \bullet HERE) \\ &= N \cdot (O \cdot (W \bullet HERE)) \\ &= N \cdot (O \cdot (W \cdot (\varepsilon \bullet HERE))) \\ &= N \cdot (O \cdot (W \cdot HERE)) \\ &= \boxed{NOWHERE} \end{aligned}$$

1.5 Induction

- Consider an arbitrary object (probably of size n)
- **Explicitly** state a **strong** inductive hypothesis (i.e. `for all k < n`)
- Explicitly handle each case for the object
- Use inductive hypothesis when it's useful
- End by pointing out you handled all cases

1.5.1 Example 1

For every string w,

$$w \bullet \varepsilon = w$$

Let w be an arbitrary string. Assume for all strings x with $|x| < |w|$, that $x \bullet \varepsilon = x$

Suppose $w = \varepsilon$. Then,

$$\begin{aligned} w \bullet \varepsilon &= \varepsilon \bullet \varepsilon && \text{since } \ w = \varepsilon \\ &= \varepsilon && \text{by definition of concatenation } \ (\bullet) \\ &= w && \text{since } \ w = \varepsilon \end{aligned}$$

1.5.2 Example 2

Suppose $w = a \bullet x$ for symbol a and string x. Prove that in all cases, $w \bullet \varepsilon = w$

$$\begin{aligned} w \bullet \varepsilon &= (a \cdot x) \bullet \varepsilon && (w = ax) \\ &= a \cdot (x \bullet \varepsilon) && (\text{by definition of } \bullet) \\ &= a \cdot x && (\text{inductive hypothesis}) \\ &= w && (w = ax) \end{aligned}$$

Therefore, in all cases, $w \bullet \varepsilon = w$

1.5.3 Example 3

Let w and z be arbitrary, upon the assumption that for every string x such that $|x| < |w|$, that $|x \bullet z| = |x| + |z|$

1. Suppose that $w = \varepsilon$. Then,

$$\begin{aligned} |w \bullet z| &= |\varepsilon \bullet z| && w = \varepsilon \\ &= |z| && \text{by definition of concatenation } \bullet \\ &= 0 + |z| && \text{arithmetic} \\ &= |\varepsilon| + |z| && \text{by definition of length of string} \\ &= |w| + |z| && w = \varepsilon \end{aligned}$$

2. Suppose that $w = a \cdot x$ for some symbol a and string x. Then,

$$\begin{aligned} |w \bullet z| &= |(a \cdot x) \bullet z| && w = a \cdot x \\ &= |a \cdot (x \bullet z)| && \text{by definition of concatenation } \bullet \\ &= 1 + |x \bullet z| && \text{inductive hypothesis} \\ &= 1 + |x| + |z| && \text{by definition of length of string} \\ &= |w| + |z| && w = a \cdot x \end{aligned}$$

2 Lecture 10 (Sep 29) - Recursion: Tower of Hanoi and Mergesort

> **Key idea** **Shrink, trust, combine:** Turn a big problem into smaller copies of the same problem. Trust that the copies get solved. Combine the answers.

The course now moves from *models of computation* (DFAs, grammars, Turing machines) to *algorithms*: how to solve problems **correctly** and **fast**.

2.1 The RealRAM Model

Same cost model as CS 225: anything a C-like language does cheaply is $O(1)$.

Operation	Cost
Arithmetic on a number	$O(1)$
Read or write `A[i]`	$O(1)$
Follow a pointer / reference	$O(1)$
Call a function (overhead)	$O(1)$

2.2 Describing Algorithms

What a "describe an algorithm" answer needs on homework and exams:

Give ✓	Don't give ×
Clear English or pseudocode	Real code (C++, Rust, Perl, …)
What each function **returns**	A correctness proof (if you follow a class pattern)
Running time: recurrence + solution	A loose bound "just to be safe"

> **Watch out** **Greedy algorithms need a formal proof:** They often look right and are wrong. Without a proof, graders assume a greedy algorithm is incorrect.

How answers are graded (target runtimes are not given):

Correct?	Speed	Credit
Yes	Faster than expected	Extra credit
Yes	Slower than expected	Big partial credit
No	Fast	Very little

> **Key idea** **Correctness first, then speed:**

2.3 Reductions and Recursion

A **reduction** solves problem A by using a solver for problem B as a **black box** (you never look inside it):

instance of A → black box solves B → answer for A

Recursion is a reduction where B is **a smaller copy of A itself**:

instance of A size n → smaller instance(s) of A → recursion fairy solves them → combine into answer

If the instance is too small to shrink, answer it directly. That is the **base case**.

2.3.1 Example: sum of an array

```
Sum(A[1..n]):
  if n = 0
    return 0                          ⟨⟨base case⟩⟩
  return Sum(A[1..n - 1]) + A[n]      ⟨⟨trust the smaller call⟩⟩
```

Sum(3 | 1 | 4) = Sum(3 | 1) +4 = 4 + 4 = 8

Blue part: handed to the recursion fairy, who returns 4. Yellow part: our own work.

2.3.2 The recursion fairy

> **Key idea** Never trace what happens **inside** a recursive call. Assume it returns the right answer. Tracing the calls of the calls overflows your brain.

Why trusting is allowed: **induction** (Lecture 01). Assume the algorithm is correct on every smaller input (inductive hypothesis). Then show it is correct on size n.

Your three jobs:

1. Every recursive call is on a **strictly smaller** instance of the **same** problem
2. Every **base case** is answered correctly
3. The **combine step** is correct, assuming the recursive calls were correct

2.4 Tower of Hanoi

Puzzle by Édouard Lucas (1883). Move the whole stack from `src` to `dst`.

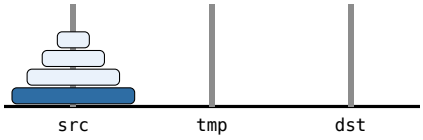


Rules:

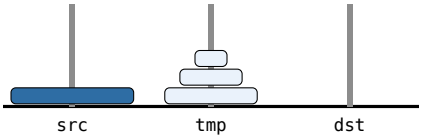
- Move **one** disk at a time, from the top of a pile to the top of another peg
- **Never** put a bigger disk on a smaller one

2.4.1 The recursive idea

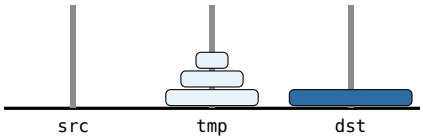
The biggest disk (dark) must move to `dst` at some point. To move it, every smaller disk must be out of the way, stacked on `tmp`.



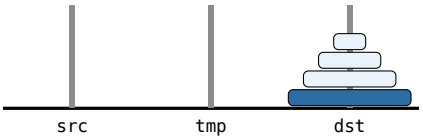
Start



Step 1: recursively move $n-1$ disks to `tmp`



Step 2: move the biggest disk to `dst`



Step 3: recursively move $n-1$ disks onto it

> **Key idea** The big disk never gets in the way: any disk may sit on it. So moving the $n-1$ small disks is **the same puzzle**, just smaller, with a different target peg.

2.4.2 Algorithm

State what the call does, so a reader can check it:

```
Hanoi(n, src, dst, tmp):   ⟨⟨move disks 1..n from src to dst⟩⟩
  if n > 0
    Hanoi(n - 1, src, tmp, dst)     ⟨⟨step 1⟩⟩
    move disk n from src to dst     ⟨⟨step 2⟩⟩
    Hanoi(n - 1, tmp, dst, src)     ⟨⟨step 3⟩⟩
```

Base case: $n = 0$, nothing to move.

2.4.3 Counting moves

Let $T(n)$ be the number of moves for n disks. Two recursive calls, plus one move:

$$T(n) = \begin{cases} 0 & \text{if } n = 0 \\ 2T(n-1) + 1 & \text{if } n > 0 \end{cases}$$

This is a **recurrence**: the cost is written in terms of the cost of smaller inputs. Compute small values and spot the pattern:

n	0	1	2	3	4	5
$T(n)$	0	1	3	7	15	31
$2^n - 1$	0	1	3	7	15	31

$$\boxed{T(n) = 2^n - 1}$$

2.4.4 Proof by induction

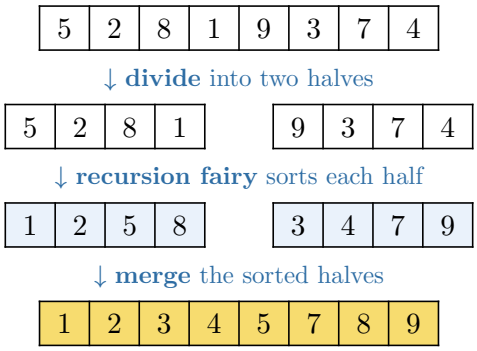
Guess-and-verify: check the guess with induction. Assume $T(k) = 2^k - 1$ for all $k < n$.

- $n = 0$: $T(0) = 0 = 2^0 - 1$ ✓
- $n > 0$:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 && \text{definition of } T \\ &= 2(2^{n-1} - 1) + 1 && \text{inductive hypothesis} \\ &= 2^n - 1 && \text{arithmetic} \end{aligned}$$

2.5 Mergesort

Sort $A[1..n]$ into increasing order. By von Neumann (1945).




```
MergeSort(A[1..n]):
  if n > 1
    m ← ⌊n/2⌋
    MergeSort(A[1..m])
    MergeSort(A[m+1..n])
    Merge(A[1..n], m)
```

Base case: $n \leq 1$. Already sorted, nothing to do.

2.5.1 Merging

Both halves are sorted, so the smallest remaining element is always at the **front** of one half. Repeatedly compare the two fronts and take the smaller one:

Left half	Right half	Take	Output so far
1 2 5 8	3 4 7 9	1	1
2 5 8	3 4 7 9	2	1 2
5 8	3 4 7 9	3	1 2 3
5 8	4 7 9	4	1 2 3 4
5 8	7 9	5	1 2 3 4 5
⋮	⋮	⋮	⋮

Each comparison places one element, so `Merge` takes $\boldsymbol{O(n)}$ time. One way to write it:

```
Merge(A[1..n], m):
  i ← 1;  j ← m + 1          ⟨⟨fronts of the two halves⟩⟩
  for k ← 1 to n
    if j > n                 ⟨⟨right half used up⟩⟩
      B[k] ← A[i];  i ← i + 1
    else if i > m            ⟨⟨left half used up⟩⟩
      B[k] ← A[j];  j ← j + 1
    else if A[i] < A[j]
      B[k] ← A[i];  i ← i + 1
    else
      B[k] ← A[j];  j ← j + 1
  for k ← 1 to n
    A[k] ← B[k]
```

2.5.2 Recurrence

Two half-size recursive calls, plus $O(n)$ for the merge:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

> **Key idea** Floors and ceilings ($\lfloor \frac{n}{2} \rfloor$ vs $\lceil \frac{n}{2} \rceil$) don't matter when the problem shrinks by a constant fraction. Just write $\frac{n}{2}$.

2.6 Recursion Trees

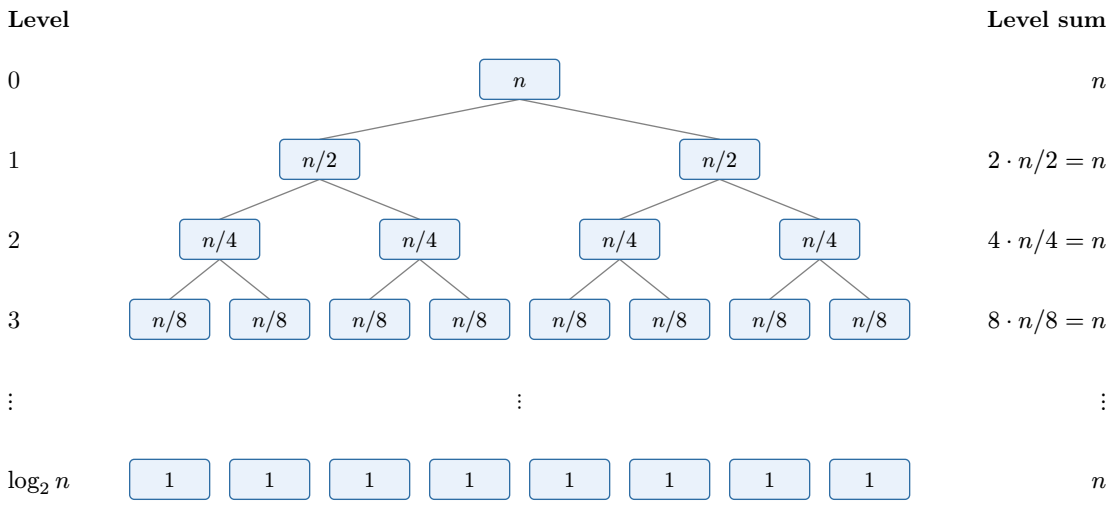
A picture of every recursive call, used to **solve** recurrences.

- One **node** per recursive call. The **root** is the first call.
- A node's **children** are the recursive calls it makes.
- A node's **value** is the work done in that call, **not counting** its children.

> **Key idea** **Total time = sum of all node values:** Add up each level (row) first, then add up the levels.

2.6.1 Mergesort

The root does n work (the merge). Each child has half the elements, so it does $\frac{n}{2}$ work. Each grandchild does $\frac{n}{4}$. And so on, down to size 1.



- **Every level sums to n:**
- **How many levels?** Each level halves the size: $n \to \frac{n}{2} \to \frac{n}{4} \to \ldots \to 1$. You can halve n only $\log_2 n$ times before reaching 1.

$$T(n) = \underbrace{n}_{\text{per level}} \times \underbrace{\log_2 n}_{\text{levels}} = \boxed{O(n \log n)}$$

The log base doesn't matter inside big-O (changing base is a constant factor).

2.6.2 The three common shapes

Look at how the level sums change from top to bottom:



total = level sum × levels
e.g. mergesort: $O(n \log n)$

root dominates
total = O(root)

leaves dominate
total = O(# leaves)

Shrinking and growing are **geometric series**: each level is the one above times a fixed ratio $\neq 1$ (for example $n, \frac{n}{3}, \frac{n}{9}, \ldots$). A geometric series is proportional to its **biggest term**, however many terms it has. Lecture 11 has examples of both.

Watch out Don't memorize a formula theorem instead. It only fits recurrences with equal-size pieces. Lecture 11's recurrences don't fit it, but recursion trees still work.

3 Lecture 11 (Oct 1) - Divide and Conquer: Quicksort and Linear Time Selection

3.1 Divide and Conquer

Mergesort is an example of a **divide and conquer** algorithm. There are no hard rules for what counts as divide and conquer, but the high-level idea is:

1. **Divide** the problem into smaller instances of the same problem
2. **Delegate** the smaller instances to the recursion fairy
3. **Combine** the solutions of the subproblems into a solution for the original instance. This step usually needs some extra work.

For mergesort: divide into two halves of size about $\frac{n}{2}$, sort both recursively, then combine with `Merge`. Combining two sorted arrays is much easier than sorting from scratch.

To analyze a divide and conquer algorithm, write a **recurrence** for the running time in terms of the smaller instances, then solve it (usually with a recursion tree).

3.2 Quicksort

Tony Hoare (1961).

1. Choose a **pivot** element. Any element is allowed; the choice is arbitrary.
2. **Partition** the array in linear time: everything smaller than the pivot moves to its left, everything bigger moves to its right.
3. Now the pivot is in its final position, but the two sides are not sorted. Neither side contains the pivot, so both are strictly smaller. Recursively sort the left side, then recursively sort the right side.

```
QuickSort(A[1..n]):
  if n > 1
    choose a pivot element A[p]
    r ← Partition(A, p)
    QuickSort(A[1..r - 1])
    QuickSort(A[r + 1..n])
```

3.2.1 Rank

`Partition` returns r, the new index of the pivot. This is the **rank** of the pivot:

- $\text{rank}(x) = 1 + (\text{number of elements smaller than } \ x)$
- Equivalently, the position of x in sorted order (1-indexed)

After partitioning, the elements smaller than the pivot are in $A[1..r-1]$ and the elements larger are in $A[r+1..n]$. Duplicate elements cause no problem: take the rank to mean the pivot's final position.

3.2.2 Partition

For completeness only. The correctness argument is subtle; see the lecture notes and Erickson's book. Move the pivot to the end, out of the way. Walk through the array; whenever an element is smaller than the pivot, swap it into the left part. Finally, swap the pivot into place.

```
Partition(A[1..n], p):
  swap A[p] ↔ A[n]
  ℓ ← 0                          ⟨⟨#items < pivot⟩⟩
  for i ← 1 to n - 1
    if A[i] < A[n]
      ℓ ← ℓ + 1
      swap A[ℓ] ↔ A[i]
  swap A[n] ↔ A[ℓ + 1]
  return ℓ + 1
```

Partitioning takes $O(n)$ time.

3.2.3 Worst case

The recursive calls do **not** have to be on halves. With pivot rank r:

$$T(n) = T(r-1) + T(n-r) + O(n)$$

If the pivot is the smallest (or largest) element, one call is empty and the other has $n-1$ elements:

$$T(n) = T(n-1) + O(n)$$

The recursion tree is a long path:

Level	**Nodes**	**Sum**
0	$\boxed{n}$	n
1	$\boxed{n-1}$	$n-1$
2	$\boxed{n-2}$	$n-2$
⋮	$\boxed{\ldots}$	⋮
$n-1$	$\boxed{1}$	1

This is an **arithmetic series**, not a geometric one. Its sum is **not** proportional to the largest term. It is proportional to (largest term) × (number of terms):

$$T(n) = \sum_{i=1}^{n} i = \boxed{O(n^2)}$$

Every simple deterministic pivot rule has bad inputs that force this: the first element, the last element, even the median of the first, middle and last elements.

3.2.4 Worst-case analysis

This course uses worst-case analysis: A stated running time must hold for every input. So all we can promise for quicksort is $O(n^2)$.

A loose bound such as $O(n!)$ is technically correct but useless, and is marked wrong. Give the bound that naturally falls out of your analysis technique.

3.2.5 Good pivots

Quicksort is fast in practice: unless someone gives it nasty inputs on purpose, the pivot rank is usually near $\frac{n}{2}$. With probability about $\frac{1}{3}$, the pivot is in the **middle third**, so both calls have at most $2n/3$ elements. In the most unbalanced such case:

$$T(n) \leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$$

Level	**Nodes**	**Sum**
0	$\boxed{n}$	n
1	$\boxed{\frac{n}{3}}$ $\boxed{\frac{2n}{3}}$	n
2	$\boxed{\frac{n}{9}}$ $\boxed{\frac{2n}{9}}$ $\boxed{\frac{2n}{9}}$ $\boxed{\frac{4n}{9}}$	n
$\vdots$	$\boxed{\ldots}$	$\leq n$

The tree is unbalanced, but every level sums to (at most) n. Some branches reach their base cases early, so lower levels may be missing nodes; that is fine for an upper bound.

The longest path follows the larger calls, multiplying by $\frac{2}{3}$ each level. The number of times you can multiply n by $\frac{2}{3}$ before reaching 1 is the number of times you can divide by $\frac{3}{2}$:

$$\text{levels} \leq \log_{3/2} n = O(\log n)$$

Any constant log base changes the result only by a constant factor, so:

$$T(n) = n \cdot \log_{3/2} n = \boxed{O(n \log n)}$$

3.3 Selection

Median: given $A[1..n]$, find the element of rank $\lceil \frac{n}{2} \rceil$.

3.3.1 Sort first

Mergesort the array and return $A[\lceil \frac{n}{2} \rceil]$. This takes $O(n \log n)$ time, and is a perfectly valid answer on a homework or exam. **If you can use something from class as a black box, do so.** But we only want one element, so try to do better.

3.3.2 Recursive idea

Pick a pivot and partition. Compare its rank r with $\lceil \frac{n}{2} \rceil$:

- $r < \lceil \frac{n}{2} \rceil$: everything left of the pivot is even smaller than the median, so the median is to the **right**. Recurse on the right side.
- $r > \lceil \frac{n}{2} \rceil$: the median is to the **left**. Recurse on the left side.
- $r = \lceil \frac{n}{2} \rceil$: the pivot is the median.

Do not pick another pivot inside the right side yourself. That is doing the recursion fairy's job, and it makes them grumpy. Just recurse.

Technical issue: inside the right subarray, the median no longer has rank $\lceil \frac{n}{2} \rceil$. Its rank is smaller relative to the subarray. So the original problem is not general enough to recurse on. Sometimes you need to solve a **more general problem** that you can call recursively.

3.3.3 The selection problem

Given an array $A[1..n]$ and an integer k with $1 \leq k \leq n$, find the element of rank k.

3.4 Quickselect

Tony Hoare, on the same page of the paper as quicksort. Think of it as a **one-armed quicksort**.

```
QuickSelect(A[1..n], k):
  if n = 1
    return A[1]
  else
    choose a pivot element A[p]
    r ← Partition(A[1..n], p)
    if k < r
      return QuickSelect(A[1..r - 1], k)
    else if k > r
      return QuickSelect(A[r + 1..n], k - r)
    else
      return A[r]
```

- $k < r$: everything right of the pivot is too big. Search the left side, still for rank k.
- $k > r$: the target is bigger than the pivot. Search the right side. The right side is missing the r smallest elements, so look for rank $k - r$.
- $k = r$: the pivot is the answer.

3.4.1 Running time

Worst case: the pivot is always the smallest or largest element. Same path-shaped recursion tree as quicksort, so $O(n^2)$.

With a pivot in the middle third, there is only **one** recursive call, on at most $2n/3$ elements:

$$T(n) \leq T\left(\frac{2n}{3}\right) + O(n)$$

Level	**Nodes**	**Sum**
0	$\boxed{n}$	n
1	$\boxed{\frac{2n}{3}}$	$\frac{2n}{3}$
2	$\boxed{\frac{4n}{9}}$	$\frac{4n}{9}$
i	$\boxed{\left(\frac{2}{3}\right)^i n}$	$\left(\frac{2}{3}\right)^i n$

The level sums form a **decreasing geometric series**. A geometric series whose ratio is not 1 is proportional to its largest term, here the root. So $T(n) = O(n)$, **if** we can always find a pivot near the middle.

3.5 Median of Medians Selection

Blum, Floyd, Pratt, Rivest and Tarjan (1970). Idea: take a representative **sample** of the array, find the sample's median recursively, and use it as the pivot. The sample's median should be close to the real median.

```
MomSelect(A[1..n], k):
  if n ≤ 25
    use brute force
  else
    m ← ⌈n/5⌉
    for i ← 1 to m
      M[i] ← MedianOfFive(A[5i − 4..5i])
    mom ← MomSelect(M[1..m], ⌊m/2⌋)
    r ← Partition(A[1..n], mom)
    if k < r
      return MomSelect(A[1..r − 1], k)
    else if k > r
      return MomSelect(A[r + 1..n], k − r)
    else
      return mom
```

1. **Big base case**: if $n \leq 25$, sort and pick directly. Constant time.
2. Split the array into blocks of 5. Find the median of each block and put it in array M of size about $\frac{n}{5}$.

3. **Recursively** find the median of M, the **median of medians** (mom). This is allowed because $\frac{n}{5} < n$.
4. Run quickselect with mom as the pivot.

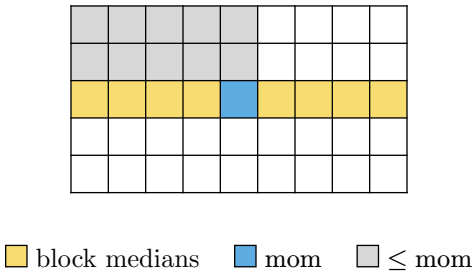
You can think of step 3 as an **approximate median fairy**, which is secretly the recursion fairy in disguise.

3.5.1 Analysis

Work outside the recursive calls: each median of five takes constant time, and there are $\frac{n}{5}$ of them; partitioning takes $O(n)$. So $O(n)$ total.

The question: is mom a good pivot? Show that neither recursive call is too big. **For the analysis only** (the algorithm does not do this), arrange A into a $5 \times \frac{n}{5}$ grid:

1. Each column is one block of 5.
2. Sort each column, smallest on top. The block medians land in the middle row.
3. Sort the columns by their medians. mom is now in the center.



- Each median to the left of mom is smaller than mom.
- Each column is sorted, so the elements above those medians are smaller still.
- There are $\frac{n}{5}$ columns, so about $\frac{n}{10}$ columns are left of mom. Each has 3 elements (the median and the two above it) smaller than mom.

So at least $3n/10$ elements are smaller than mom, and **at most $7n/10$ elements are larger**. Symmetrically, at most $7n/10$ elements are smaller. The other cells could go either way, but they are not needed. Either recursive call has at most $7n/10$ elements:

$$T(n) \leq O(n) + T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right)$$

Level	Nodes	Sum
0	$\boxed{n}$	n
1	$\boxed{\frac{n}{5}}$ $\boxed{\frac{7n}{10}}$	$\frac{9n}{10}$
2	$\boxed{\frac{n}{25}}$ $\boxed{\frac{7n}{50}}$ $\boxed{\frac{7n}{50}}$ $\boxed{\frac{49n}{100}}$	$\frac{81n}{100}$
i	$\boxed{\ldots}$	$\left(\frac{9}{10}\right)^i n$

$\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, so the level sums form a **decreasing geometric series**. The number of levels does not matter; the sum is proportional to the root:

$$T(n) = \boxed{O(n)}$$

This is optimal: any algorithm must look at every element.

3.5.2 Why blocks of 5?

5 is the smallest odd block size for which the analysis works (7 also works). With blocks of 3, the same argument gives

$$T(n) \leq O(n) + T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right)$$

Every level sums to n, which is the good-pivot quicksort tree: $O(n \log n)$, no better than sorting.

3.5.3 Takeaways

- Divide and conquer can use **unexpected** subproblems, such as recursing on a sample just to find a pivot.
- Subproblems can be **unbalanced**. Use recursion trees to analyze them.

3.6 Preview: Fast Multiplication

Next lecture: multiply two n-digit numbers with $O\left(n^{\log_2 3}\right) \approx O\left(n^{1.59}\right)$ single-digit multiplications. Another divide and conquer algorithm, which splits each number's digits into two halves.

4 Lecture 12 (Oct 6) - Divide and Conquer: Multiplication; Backtracking: Text Segmentation

4.1 Fast Multiplication

Grade-school (lattice) multiplication of two n-digit numbers multiplies each digit of one number by the whole other number, then adds the results. Each digit costs $O(n)$, and there are n digits, so the total is $O(n^2)$. Can divide and conquer do better?

4.1.1 Cost model

Normally we use the RealRAM model, where arithmetic on any number costs $O(1)$. That defeats the point here. Instead, **count single-digit operations**: adding, subtracting or multiplying one pair of digits.

With this model, some operations on n-digit numbers are cheap:

- Adding or subtracting two n-digit numbers: $O(n)$ (school method)
- Multiplying by a power of 10 (padding with zeros): $O(n)$
- Splitting a number into its top and bottom digits: $O(n)$

Multiplication is the only hard part.

4.1.2 Split into halves

Let $m = \lceil \frac{n}{2} \rceil$. Split each input into its m most significant digits and its m least significant digits:

$$x = a \cdot 10^m + b \qquad y = c \cdot 10^m + d$$

Each of a, b, c, d has at most m digits. Multiply out (FOIL):

$$xy = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd$$

This reduces one n-digit product to **four** m-digit products.

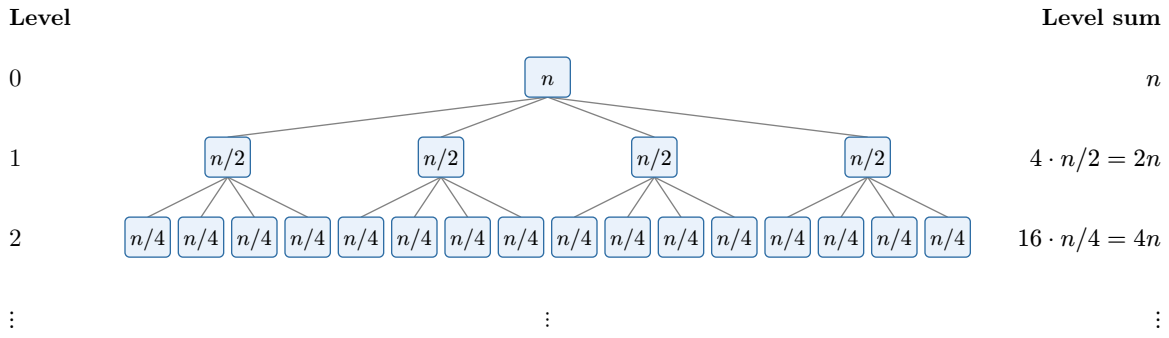
```
SplitMultiply(x, y, n):
  if n = 1
    return x · y                    ⟨⟨times table⟩⟩
  else
    m ← ⌈n/2⌉
    a ← ⌊x / 10^m⌋;  b ← x mod 10^m
    c ← ⌊y / 10^m⌋;  d ← y mod 10^m
    e ← SplitMultiply(a, c, m)
    f ← SplitMultiply(b, d, m)
    g ← SplitMultiply(b, c, m)
    h ← SplitMultiply(a, d, m)
    return 10^(2m)·e + 10^m·(g + h) + f
```

The `mod` and `/` by 10^m only chop the digit string, and the final line is padding and addition. All of that is $O(n)$.

4.1.3 Analysis

Let $T(n)$ be the number of single-digit operations. Four recursive calls of size $\frac{n}{2}$, plus $O(n)$ extra work:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$



The level sums $n, 2n, 4n, \ldots$ form an **increasing geometric series** (ratio 2). A geometric series with ratio $\neq 1$ is proportional to its largest term. Here the largest term is the **bottom** level.

> **Key idea** **Increasing series, count the leaves:** Every leaf is a base case, so it does $O(1)$ work. The bottom level sum is $O(\#\text{ leaves})$.

- Each level has 4 times as many nodes as the one above.
- The size halves each level, so there are $\log_2 n$ levels.

$$\#\text{ leaves} = 4^{\log_2 n}$$

To simplify, use this log identity. You may swap the base and the argument of the log:

$$a^{\log_b c} = c^{\log_b a}$$

$$T(n) = O\left(4^{\log_2 n}\right) = O\left(n^{\log_2 4}\right) = \boxed{O(n^2)}$$

No improvement over grade school. Until 1960, people believed $O(n^2)$ was the best possible.

4.2 Karatsuba's Algorithm

Karatsuba (1960) noticed that the middle term needs only **one** new product. Add and subtract ac and bd:

$$ad + bc = ac + bd - (a - b)(c - d)$$

Check: $(a-b)(c-d) = ac - ad - bc + bd$, so $ac + bd - (a-b)(c-d) = ad + bc$.

The products ac and bd are already needed. So three recursive calls suffice: ac, bd and $(a-b)(c-d)$.

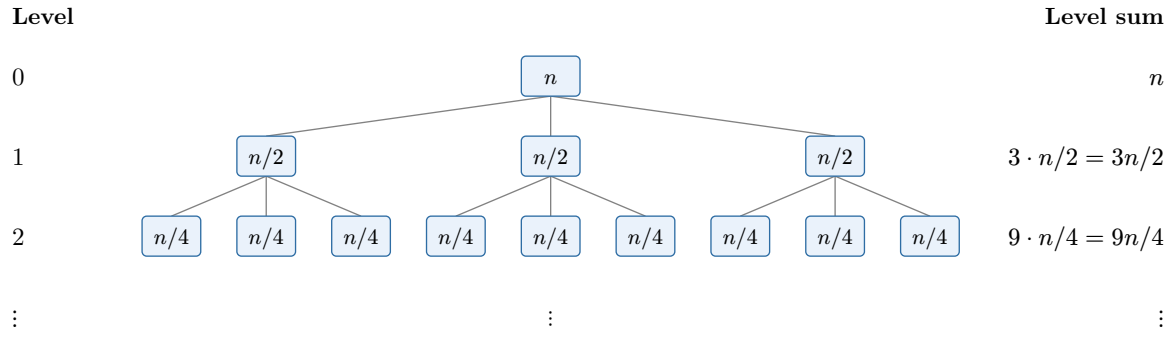
```
FastMultiply(x, y, n):
  if n = 1
    return x · y
  else
    m ← ⌈n/2⌉
    a ← ⌊x / 10^m⌋;  b ← x mod 10^m
    c ← ⌊y / 10^m⌋;  d ← y mod 10^m
    e ← FastMultiply(a, c, m)
    f ← FastMultiply(b, d, m)
    g ← FastMultiply(a − b, c − d, m)
    return 10^(2m)·e + 10^m·(e + f − g) + f
```

> **Watch out** $a - b$ and $c - d$ can be **negative**. Both still have at most m digits. Multiply their absolute values recursively, then fix the sign.

4.2.1 Analysis

Three recursive calls, still on size $\frac{n}{2}$, still $O(n)$ extra work:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$



The level sums still increase geometrically (ratio $\frac{3}{2}$), so count the leaves. The depth is still $\log_2 n$, but each level now has 3 times as many nodes:

$$T(n) = O\left(3^{\log_2 n}\right) = O\left(n^{\log_2 3}\right) = \boxed{O(n^{1.585})}$$



> **Key idea** **Keep the number of recursive calls small:** The sizes shrink the same way in both algorithms. One call fewer turns n^2 into $n^{1.585}$, because the leaf count is $(\#\text{ calls})^{\text{depth}}$.

4.2.2 Later history

Chop the numbers into more pieces and combine the products more cleverly, and the time keeps dropping:

- 1971 (Schönhage and Strassen): $O(n \log n \log\log n)$
- 2019 (Harvey and van der Hoeven): $O(n \log n)$

The 2019 algorithm is purely theoretical. It only beats the 1971 one on numbers with more digits than there are particles in the universe.

4.2.3 Recursion tree shapes, all three seen

Level sums	Total	Example
Equal	level sum $\times$ # levels	Mergesort: $O(n \log n)$
Decreasing geometric	$O(\text{root})$	Mom select: $O(n)$
Increasing geometric	$O(\#\text{ leaves})$	Karatsuba: $O\left(n^{\log_2 3}\right)$

4.3 Backtracking

The next four lectures cover a second recursive strategy. Today is **only about correctness**. Thursday shows how to make these algorithms fast.

Many problems need a **sequence of decisions**:

- **List** every way to do a task
- Find the **best** way to do a task
- Decide whether the task is **possible** at all

> **Key idea** **Make one decision, try every option, recurse on the consequences:** Let the recursion fairy make all the other decisions.

How to use the recursive results depends on the goal:

Goal	Combine the recursive results by
List all solutions	Collect the results of every call
Find the best solution	Keep the best result
Is it possible?	Return true if **any** call returns true (OR)

4.4 Text Segmentation

Input: a string $A[1..n]$ with the spaces removed, and a black-box function IsWord that says whether a string is a word.

Output: true if A can be partitioned into substrings that are all words, false otherwise.

Ancient Latin was often written with no spaces, so this is a real problem. What counts as a "word" is the job of IsWord, not ours.

4.4.1 Decisions

Choose the words one at a time, left to right: what is the **next** word?

Example: `BLUESTEMUNITROBOTHEARTHANDSATURNSPIN`. Suppose the first four words are `BLUE STEM UNIT ROBOT`. The next word could be any of:

Next word	Left to segment
HE	ARTHANDSATURNSPIN
HEAR	THANDSATURNSPIN
HEART	HANDSATURNSPIN
HEARTH	ANDSATURNSPIN

We don't know which one is right, so **try all four**. For each, recursively ask whether the rest can be segmented. Any one success means the whole string works.

4.4.2 The recursive subproblem

What does the recursion fairy need to know? **Not** the words chosen so far. Only the **characters that remain** matter. Many different earlier choices can lead to the same remaining suffix, and the answer for that suffix is the same each time.

> **Key idea** Finding the right recursive subproblem is the main goal. Here it is: "can this **suffix** be segmented?" The algorithm follows almost automatically.

Base case: the empty string can always be segmented, into zero words. So return true.

```
Splittable(A[1..n]):
  if n = 0
    return True
  for i ← 1 to n
    if IsWord(A[1..i])
      if Splittable(A[i + 1..n])
        return True
  return False
```

Try every possible first word $A[1..i]$. If it is a word **and** the rest $A[i+1..n]$ can be segmented, succeed. If no first word works, fail.

Watch out **This is not greedy:** A greedy algorithm commits to one choice, for example "always take the shortest first word". That is probably wrong for some inputs. Backtracking tries **every** choice, so it is always correct.

4.4.3 Running time

Correct, but slow: it can make $O(2^n)$ calls to IsWord. Thursday shows how to make it fast.

4.4.4 Pass indices, not arrays

Copying the suffix into every recursive call is wasteful. Make $A[1..n]$ a **global** array. Every subproblem is a suffix, so one index identifies it: Splittable(i) asks whether $A[i..n]$ can be segmented.

```
Splittable(i):                          ⟨⟨A[1..n] is global⟩⟩
  if i > n
    return True
  for j ← i to n
    if IsWord(i, j)
      if Splittable(j + 1)
        return True
  return False
```

4.4.5 As a recurrence

The input is one integer and the output is a boolean. So Splittable is a mathematical function, and a recurrence can define it instead of pseudocode. Define:

- IsWord(i, j): true if and only if the substring $A[i..j]$ is a word
- Splittable(i): true if and only if the suffix $A[i..n]$ can be segmented

$$\text{Splittable}(i) = \begin{cases} \text{True} & \text{if } i > n \\ \bigvee_{j=i}^{n} (\text{IsWord}(i, j) \wedge \text{Splittable}(j+1)) & \text{otherwise} \end{cases}$$

The big $\vee$ tries every possible end j of the first word. **The answer to the original problem is Splittable(1).**

This recurrence describes the same algorithm as the pseudocode. Thursday turns this form into a fast algorithm almost mechanically.

4.5 Summary: the Backtracking Recipe

1. Think of the problem as a **sequence of decisions** (here: the next word).
2. Find the **recursive subproblem**: what the recursion fairy must know to finish (here: the remaining suffix, as one index i).
3. Try **every option** for one decision, and recurse on each.

4. Pick a **base case** where no decision is left (here: the empty suffix is true).
5. Write it as a **recurrence** on indices, and say which call answers the original problem.

> **Watch out** Don't skip steps 1 and 2, even after Thursday. The speedup only works once you have a correct recurrence to apply it to.

More backtracking examples are in the course notes and in Erickson's book, chapter 2.

5 Lecture 13 (Oct 8) - Dynamic Programming: Fibonacci Numbers, Faster Text Segmentation

> **Key idea** **Dynamic programming = backtracking + bottom-up evaluation:** First get a correct recurrence (hard, creative). Then fill a table in the right order (easy, mechanical).

5.1 Fibonacci Numbers

For all integers $n \geq 0$:

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$

Virahanka described these numbers around 800 CE. Fibonacci described them again in 1202, and his name stuck.

5.1.1 Recursive algorithm

The definition is already an algorithm:

```
RecFibo(n):
  if n = 0
    return 0
  else if n = 1
    return 1
  else
    return RecFibo(n - 1) + RecFibo(n - 2)
```

Count additions. Two recursive calls, of sizes $n-1$ and $n-2$, plus one addition:

$$T(n) = T(n-1) + T(n-2) + 1, \qquad T(0) = T(1) = 1$$

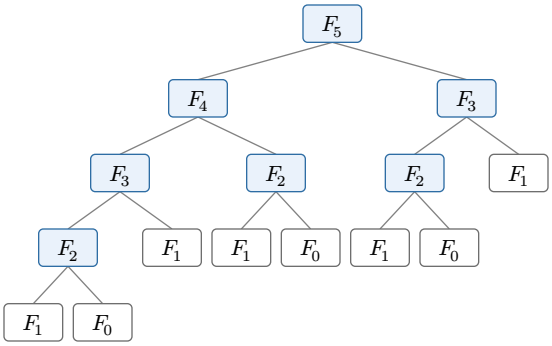
This is the Fibonacci recurrence again, plus 1. Its solution:

$$T(n) = 2F_{n+1} - 1 = \Theta(\varphi^n), \qquad \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Exponential time. You don't need to know how to solve this recurrence.

5.1.2 Why so slow?

The call tree for F_5 (each node shows the value it computes):



The same values are computed again and again:

Call	F_4	F_3	F_2	F_1	F_0
Times computed	1	2	3	5	3

The number of repeats also grows exponentially with n. Almost all of the work is wasted.

5.2 Memoization

Michie (1968) proposed **memo functions**. A memo function stores each result. When it is called again with the same input, it returns the stored result and does not compute it again. Turning a function into a memo function is called **memoization** (not a typo for "memorization": think **memo**, a written note).

How to memoize:

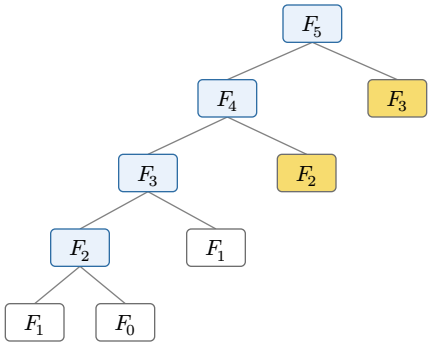
1. Keep a **global** data structure outside the function calls.
2. On each call, check whether the answer is already stored. If so, use it.
3. If not, compute it from scratch and **store it** before using it.

> **Watch out** **Use an array, not a hash table:** Array access is truly $O(1)$. Hash tables are $O(1)$ only **in expectation**, which adds a way for your running time (or your analysis) to go wrong. Usually a plain array is all you need.

```
MemFibo(i):                          ⟨⟨F[2..n] is global⟩⟩
  if i = 0
    return 0
  else if i = 1
    return 1
  else
    if F[i] is undefined
      F[i] ← MemFibo(i − 1) + MemFibo(i − 2)
    return F[i]
```

5.2.1 Analysis

The call tree for F_5 with memoization. Yellow nodes return immediately with an array lookup:

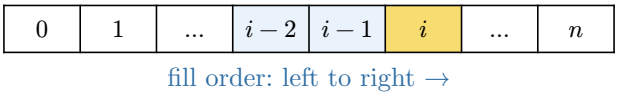


The tree is now a **path** with small leaves hanging off it. Look at what actually happens: the algorithm fills the array **in order**, $F[2], F[3], ..., F[n]$, left to right. Each entry is filled once, with one addition of two values that are already known.

$$O(n) \text{ entries} \times O(1) \text{ each} = \boxed{O(n)}$$

5.3 Iterative Fibonacci

The memoized algorithm really just fills the array from small to big. So do that **explicitly**, with a loop and no recursion:




```
IterFibo(n):
  F[0] ← 0
  F[1] ← 1
  for i ← 2 to n
    F[i] ← F[i − 1] + F[i − 2]
  return F[n]
```

One loop, so $O(n)$ additions, and easy to analyze. It is also faster in practice: no call stack, no function-call overhead.

5.4 Dynamic Programming

IterFibo is the first example of **dynamic programming**. It is more than "write a recursive algorithm and memoize it", and it is **not** "make a table and fill it in". It has two parts, and **both are required**:

Part	What	Purpose
1 · Formulate recursively	Backtracking: a recursive solution as a recurrence	**Correctness**. The hard, creative part.
2 · Build bottom-up	Fill a data structure with subproblem answers, usually iteratively	**Speed**. Mostly mechanical with practice.

Name: Bellman popularized the method in the mid-1950s while working on military scheduling problems. "Programming" means planning a schedule, as in TV programming, not writing code. "Dynamic" because the table changes over time as it fills. Today it is probably the most important technique for designing correct, efficient algorithms.

> **Watch out** **Write out every step on homework and exams:** The steps are in Erickson's *Algorithms* (chapter 3), on the back of the lab handout, and in the grading rubrics. Skipping steps costs points, and it is how incorrect algorithms happen.

5.5 Faster Text Segmentation

Recall the problem: given $A[1..n]$ and IsWord, decide whether A can be split into words. Now go through the whole process.

5.5.1 Part 1: Formulate recursively

Use backtracking to find the subproblem: a sequence of decisions (the next word), try every option, recurse on what is left.

(a) Specify the subproblem in English: Give a **closed definition** of what the function returns, like good documentation for a library function.

$$\text{Splittable}(i) = \text{True if and only if the suffix } A[i..n] \text{ can be split into words}$$

The definition may refer to the original input A. It must **not** say how to compute the value, or how the value gets used. It is a well-defined problem on its own. (Same idea as context-free grammars: you described the language of each nonterminal on its own, without the other nonterminals.)

(b) Give a recurrence: With $\text{IsWord}(i, j)$ true if and only if $A[i..j]$ is a word:

$$\text{Splittable}(i) = \begin{cases} \text{True} & \text{if } i > n \\ \bigvee_{j=i}^{n} (\text{IsWord}(i, j) \wedge \text{Splittable}(j + 1)) & \text{otherwise} \end{cases}$$

Prefer a recurrence over recursive pseudocode. It makes part 2 much cleaner.

(c) Say how to answer the original problem: Call Splittable(1).

Key idea Part 1 is the hard part, and part 2 means nothing without it. Everything after this point is mechanical.

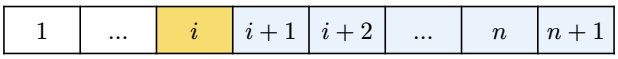
5.5.2 Part 2: Build solutions bottom-up

(a) Identify the subproblems: List every way the function can be called. Calls always move to a larger index and stop just past n. So the subproblems are Splittable(i) for all $1 \le i \le n+1$. (For Fibonacci: F_i for all $0 \le i \le n$.)

(b) Choose a memoization data structure: Usually an array, sometimes multidimensional. It must hold every subproblem: here an array SplitTable[$1..n+1$].

Watch out **Pick descriptive names:** Name the array after the function it stores. Never call it `DP`, `OPT`, `ans` or `table`. The grader must be able to tell what is stored. Bad names lose points.

(c) Identify dependencies: To fill entry i, which other entries must already be known? Read it off the recurrence. Splittable(i) needs Splittable($j+1$) for every $j \ge i$: **every entry to its right**. Draw the array with arrows into entry i:



← fill order: right to left

(d) Find an evaluation order: Fill each entry only after everything it depends on. Dependencies point to **bigger** indices, so go from big to small: i from $n+1$ down to 1. Pseudocode ("for $i \leftarrow n+1$ down to 1"), words ("right to left"), or a big arrow on the drawing all count.

(e) Write the algorithm: Wrap the recurrence in loops that follow the evaluation order. Copy the recurrence as closely as possible:

```
FastSplittable(A[1..n]):
  for i ← n + 1 down to 1
    if i > n
      SplitTable[i] ← True
    else
      SplitTable[i] ← False
      for j ← i to n
        if IsWord(i, j) and SplitTable[j + 1]
          SplitTable[i] ← True
  return SplitTable[1]
```

- **No "is it undefined?" check is needed:** The evaluation order guarantees that SplitTable[$j+1$] is already filled when it is read.
- Erickson fills the base cases first, then loops over the rest. Checking the base case **inside** the loop is also fine, and copies the recurrence more directly.

(f) Analyze the running time: (Number of subproblems) × (time per subproblem). There are $n+1$ subproblems, and each tries $O(n)$ values of j. Or just read the two nested loops:

$$\boxed{O(n^2) \text{ calls to IsWord}}$$

Exponential before, quadratic now. (If IsWord took time linear in the word's length, the total would be more, perhaps $O(n^3)$.)

> **Key idea** **On homework and exams the pseudocode is optional:** The subproblem definition, recurrence, evaluation order and running time already describe the whole algorithm. Check the rubric for what is required.

5.6 Summary: the Dynamic Programming Recipe

Step	**Text segmentation**
1a. Define the subproblem in English	Splittable(i): can $A[i..n]$ be split?
1b. Recurrence (with base cases)	Big OR over the first word $A[i..j]$
1c. Which call answers the problem	Splittable(1)
2a. All subproblems	$1 \le i \le n+1$
2b. Data structure	Array SplitTable$[1..n+1]$
2c. Dependencies	Entry i needs every entry $> i$
2d. Evaluation order	i from $n+1$ down to 1
2e. Algorithm (optional)	Loops around the recurrence
2f. Running time	$O(n^2)$ calls to IsWord

Next: the lab turns five backtracking solutions for sequence problems into dynamic programming algorithms. Next week covers more backtracking patterns that lead to good dynamic programming algorithms.

6 Lecture 15 (Oct 15) - Dynamic Programming on Trees: Woodcutting, Maximum Independent Set

So far, dynamic programming has read one or two arrays from one end to the other. Today the recursion has a different shape: it builds a **tree**. This is the last lecture on dynamic programming for its own sake.

> **Key idea** **Make one decision, and the rest splits into independent pieces:** Solve each piece recursively and add up the costs. Woodcutting splits a plank into two subplanks. Independent set splits a tree into its subtrees.

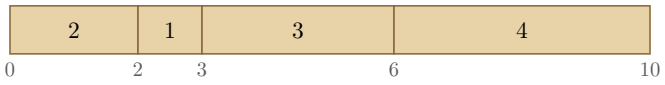
6.1 Woodcutting

6.1.1 The problem

A plank of wood has $n-1$ marks on it. It must be cut at every mark, and only at the marks, into n final pieces. Only one company owns a saw big enough. It does **one cut at a time** and charges the **length of the plank it is cutting**. Each cut gives two smaller planks, which you hand back for more cuts.

Input: an array Len[1..n] of the final piece lengths, in order from left to right. **Output:** the minimum total cost to make all the cuts.

Example: A plank of length 10 with marks at 2, 3 and 6, so $n = 4$ and Len $= [2, 1, 3, 4]$:



The first cut always costs \$10, whichever mark it uses. After that, the order matters:

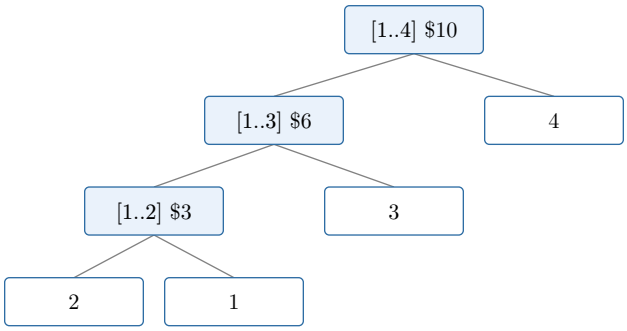
Strategy	Cuts, in order (plank length paid)	Total
Left to right	at 2 (10), at 3 (8), at 6 (7)	\$25
Middle first	at 3 (10), then at 2 (3) and at 6 (7)	\$20
Right to left	at 6 (10), at 3 (6), at 2 (3)	**\$19**

> **Watch out** **The most balanced cut is not always best:** Right to left wins here: the long piece of length 4 is cut off first, so you stop paying for it early.

6.1.2 Part 1: Formulate recursively

Decision: where to make the first cut. It costs the length of the whole plank. It leaves a left subplank and a right subplank, and these are **independent** problems: how you cut one does not change the cost of the other.

The cuts form a **binary tree**. The root is the whole plank, and its two children are the two subplanks. Choosing the first cut means choosing the root. The optimal strategy for the example:



Internal nodes: the pieces on that plank and the cost of its cut. Leaves: final piece lengths.

Every subplank holds a **contiguous** run of final pieces, so a pair of indices names it.

Subproblem: For $1 \le i \le k \le n$:

$$\text{Cutting}(i, k) = \text{minimum cost to cut the plank made of pieces Len}[i..k] \text{ into those pieces}$$

Recurrence: Guess the index r of the first piece in the right subplank. The right subplank must not be empty, and neither may the left one, so $i < r \le k$:

$$\text{Cutting}(i,k) = \begin{cases} 0 & \text{if } i = k \\ \sum_{j=i}^{k} \text{Len}[j] + \min_{i<r\le k} \left(\text{Cutting}(i, r-1) + \text{Cutting}(r, k)\right) & \text{otherwise} \end{cases}$$

The sum is the cost of the first cut (the length of the whole subplank). The min tries every place for that cut. A single piece ($i = k$) needs no cuts.

Original problem: Cutting$(1, n)$.

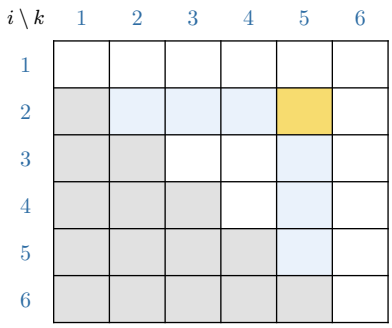
> **Key idea** **Try every root. Don't try to be clever:** You don't know which cut goes first, so try all of them, and let the recursion handle each side.

6.1.3 Part 2: Build solutions bottom-up

Subproblems: Cutting(i, k) for all $1 \le i \le k \le n$.

Data structure: a 2D array Cutting$[1..n, 1..n]$. Only the entries with $i \le k$ are used.

Dependencies: Entry (i, k) reads Cutting$(i, r-1)$ for $i \le r-1 < k$: the **same row, to the left**. It also reads Cutting(r, k) for $i < r \le k$: the **same column, below**. For $n = 6$ and entry $(2, 5)$:



Rows are i, columns are k. Yellow: the entry being filled. Blue: the entries it reads. Gray: unused ($i > k$).

Evaluation order: Any order that fills the row to the left and the column below first. Three choices work:

Order	Loops	Verdict
Row by row	i from n **down** to 1; inside, k from i **up** to n	Good
Column by column	k from 1 **up** to n; inside, i from k **down** to 1	Good
Diagonal by diagonal	Short subplanks first ($k - i = 0, 1, 2, ...$)	Correct, but the indices are easy to get wrong

> **Watch out** **The two loops go in opposite directions**: one counts up, one counts down. Read each direction off the dependency drawing. Don't assume both go the same way.

Running time: Each entry adds up to n lengths and tries up to n values of r, so $O(n)$ time per entry. (Erickson precomputes the sums, but it doesn't change the answer here.) There are $O(n^2)$ entries:

$$O(n^2) \text{ subproblems} \times O(n) \text{ each} = \boxed{O(n^3)}$$

On homework, you can stop here. The pseudocode is just nested loops, and it is no easier to read than the steps above. For reference (Erickson's book names the array `MinCost`):

```
Woodcut(Len[1..n]):
  for i ← n down to 1
    Cutting[i, i] ← 0
    for k ← i + 1 to n
      total ← Len[i] + Len[i + 1] + ... + Len[k]
      best ← ∞
      for r ← i + 1 to k
        best ← min(best, Cutting[i, r − 1] + Cutting[r, k])
```

```
    Cutting[i, k] ← total + best
return Cutting[1, n]
```

The example, filled in: Len $= [2, 1, 3, 4]$, row by row from the bottom:

$i \setminus k$	1	2	3	4
1	0	3	9	19
2		0	4	12
3			0	7
4				0

For example, $\text{Cutting}(1, 4) = 10 + \min(0 + 12, 3 + 7, 9 + 0) = 19$. The best first cut is $r = 4$: cut off the last piece, as in the right-to-left strategy.

6.2 Maximum Independent Set in Trees

6.2.1 The problem

Let $G = (V, E)$ be a graph. An **independent set** is a subset $S \subseteq V$ such that **no edge has both endpoints in S**. The goal: find the largest one.

In a general graph this is **NP-hard** (later in the semester). There is probably no polynomial-time algorithm, not even an $O(n^{100})$ one. So restrict the input to a **rooted tree** T with n nodes and root r. Nodes may have any number of children: it is not necessarily a binary tree. Rooted trees are recursive: a root, plus a rooted subtree for each child.

6.2.2 Part 1: Formulate recursively

Decision: is the root in the set or not?

- **Root not in the set:** Its children are all allowed. There are no edges between different subtrees, so take the best independent set of each child's subtree and combine them.
- **Root in the set:** Every child shares an edge with the root, so no child is allowed. The children's subtrees below them are free: take the best set in each **grandchild's** subtree, plus the root.

The recursive calls are always on subtrees. Name a subtree by its root, as with binary search trees in CS 225.

Subproblem: For each node v:

$$\text{MIS}(v) = \text{the size of the largest independent set in the subtree rooted at } v$$

Notation: $w \downarrow v$ means "w is a child of v".

Recurrence:

$$\mathrm{MIS}(v) = \max\left\{ \sum_{w \downarrow v} \mathrm{MIS}(w), \quad 1 + \sum_{w \downarrow v} \sum_{x \downarrow w} \mathrm{MIS}(x) \right\}$$

No separate base case is needed. At a leaf both sums are empty, so $\mathrm{MIS}(\text{leaf}) = \max\{0, 1\} = 1$.

Original problem: $\mathrm{MIS}(r)$.

6.2.3 Part 2: Build solutions bottom-up

Subproblems: one per node v.

Data structure: The subproblems are named by nodes, not integers, so an array no longer fits. Store each answer in a **field on the node**: v.MIS. (Same idea as a distance field on each vertex in Dijkstra's algorithm.)

Dependencies: v reads its children and grandchildren: nodes **below** v.

Evaluation order: postorder. It visits all of a node's descendants before the node.

> Key idea **"Evaluate in postorder" is a complete answer:** Computing a postorder is a black-box algorithm you may use. Compute it, then loop over the nodes in that order. The same answer works for directed acyclic graphs next week.

Running time: The usual (number of subproblems) $\times$ (time per subproblem) gives a bound that is far too big. The time per node varies a lot: one node may have $n/2$ children while most have none. Max time $\times$ count gives $O(n^2)$.

Instead, **add up the work and charge it to the node being read**. Node v costs $O(1 + \#\text{children} + \#\text{grandchildren})$. Each node is

- a child of at most one node, so it is read as a child at most once in total;
- a grandchild of at most one node, so it is read as a grandchild at most once in total.

So all the reads add up to $O(n)$, and the total is

$$O(n) \text{ for the nodes} + O(n) \text{ child reads} + O(n) \text{ grandchild reads} = \boxed{O(n)}$$

This is only an analysis trick: the algorithm does nothing extra. (Another view: there is one child read per edge, and a tree has $n - 1$ edges.)

> Watch out **For tree and graph DP, don't multiply the maximum time per subproblem by the number of subproblems.** Add the work over all nodes, and charge each read to the node (or edge) it touches.

6.2.4 A cleaner version: two subproblems

Split the subproblem in two, by whether v is in the set:

$$\text{MISyes}(v) = \text{largest independent set in } v\text{'s subtree that must contain } v$$
$$\text{MISno}(v) = \text{largest independent set in } v\text{'s subtree that must not contain } v$$

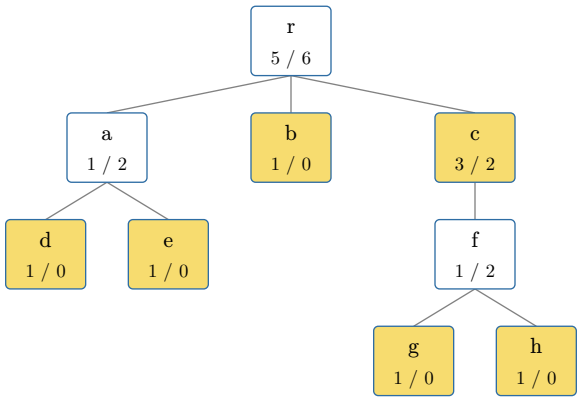
If v is in, no child is: there is no decision. If v is out, each child is free: take the better of in and out.

$$\text{MISyes}(v) = 1 + \sum_{w \downarrow v} \text{MISno}(w)$$
$$\text{MISno}(v) = \sum_{w \downarrow v} \max\{\text{MISyes}(w), \text{MISno}(w)\}$$

The two recurrences call each other, but there are no grandchildren sums. The answer is $\max\{\text{MISyes}(r), \text{MISno}(r)\}$. Store two fields on each node. Each node still depends only on its children, so evaluate in postorder: $O(n)$ time.

```
TreeMIS(T):
  for each node v in postorder
    v.yes ← 1
    v.no ← 0
    for each child w of v
      v.yes ← v.yes + w.no
      v.no ← v.no + max(w.yes, w.no)
  return max(r.yes, r.no)
```

Example: Each node shows MISyes / MISno. Yellow nodes form a maximum independent set of size $\max\{5, 6\} = 6$:



For example, $\text{MISyes}(c) = 1 + \text{MISno}(f) = 3$, and $\text{MISno}(r) = 2 + 1 + 3 = 6$ (the better value of a, b and c). The set: leave out r, a and f; take b, c, d, e, g, h.

6.3 Summary

Step	**Woodcutting**	**Tree MIS**
Decision	Where the first cut goes	Is the root in the set?
Subproblem	Cutting(i, k): pieces $i..k$	MIS(v): subtree at v
Original problem	Cutting$(1, n)$	MIS(r)
Data structure	2D array Cutting$[1..n, 1..n]$	Field v.MIS on each node
Dependencies	Same row to the left, same column below	Children and grandchildren
Evaluation order	i down from n; k up from i	Postorder
Running time	$O(n^2) \times O(n) = O(n^3)$	$O(n)$, by charging reads to nodes

7 Lecture 16 (Oct 20) - Graphs: Definitions, Representations, Search, Reductions

For the next two or two and a half weeks: fundamental **graph algorithms**. You will rarely design a new graph algorithm on homework. Instead you will **reduce** problems to these algorithms: build a graph, run a known algorithm on it, and read off the answer. Lecture explains how the algorithms work; homework uses them.

7.1 What Is a Graph?

A drawing of dots and curves is only a picture. Two very different drawings can show the same graph. The definition is more abstract:

A **graph** is an ordered pair $G = (V, E)$ of two sets.

- V is any non-empty, finite set: the **vertices** (or nodes). Same idea as an alphabet.
- E is the set of **edges** (or arcs). What an edge is depends on the kind of graph:

Kind	**An edge is**	**Written**
Directed	an ordered pair (u, v) with $u, v \in V$	$u \to v$
Undirected	an unordered pair $\{u, v\}$: a set of exactly two vertices	uv

The short notation keeps descriptions of edge sets readable without nested brackets.

7.1.1 Vertices can be anything

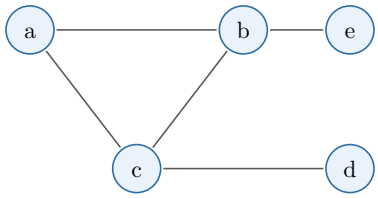
V is any finite set, so you can make the **objects of a problem** into vertices. This is the key to reductions.

Graph	**Vertices**	**Edges**
Intersection graph	Geometric objects (segments, disks)	Pairs of objects that intersect
Dependency graph	Distinct subproblems of a recursive algorithm	$u \to v$ if u calls v directly. Fibonacci: $F_6 \to F_5$ and $F_6 \to F_4$
Configuration graph	States of a game, puzzle or machine (for example, every legal placement of 4 Tower of Hanoi disks)	Pairs one move apart. Directed if a move can't be undone
NFA state graph	States	Possible transitions on one symbol

"Can this puzzle get from configuration A to configuration B?" is exactly **"is there a path from A to B in the configuration graph?"** The path is the sequence of moves. Dependency graphs come back soon, as another way to see dynamic programming.

7.2 Representing a Graph

Assume G is **simple**: no parallel edges (never two edges from u to v, or two edges with endpoints u and v). Example, undirected:



	a	b	c	d	e
a	0	1	1	0	0
b	1	0	1	0	1
c	1	1	0	1	0
d	0	0	1	0	0
e	0	1	0	0	0

a	b → c
b	a → c → e
c	a → b → d
d	c
e	b

The graph, its adjacency matrix, and its adjacency list.

7.2.1 Adjacency matrix

A $|V| \times |V|$ array, indexed by vertices. Entry (u, v) is 1 if $u \to v \in E$ (directed) or $uv \in E$ (undirected), and 0 otherwise. An undirected edge gives **two** 1s, at (u, v) and (v, u).

7.2.2 Adjacency list

A 1D array indexed by vertices. Entry u is a list of its **neighbors**: every v with $u \to v \in E$ (directed) or $uv \in E$ (undirected). A linked list is fine; nothing fancier is ever needed.

Operation	**Adjacency matrix**	**Adjacency list**
Is $u \to v$ an edge?	$O(1)$	$O(\deg(u))$: walk u's list
List the neighbors of u	$O(\|V\|)$: walk the whole row	$O(1)$ per neighbor
Space	$O(\|V\|^2)$, even with no edges	$O(\|V\| + \|E\|)$

From now on, assume adjacency lists and give running times in terms of $|V|$ and $|E|$. Graph algorithms almost always ask "what are u's neighbors?" and almost never ask "is uv an edge?", so the slow edge test doesn't matter.

An intersection graph behaves like an **implicit** adjacency matrix: you can test whether two objects intersect in $O(1)$, but you can't list an object's neighbors without checking every other object.

7.3 Reachability and Whatever-First Search

Reachability: Given a graph $G = (V, E)$ (directed or undirected) and a vertex s, find every vertex v that s can **reach**: there is a path from s to v.

- **Directed:** the path must start at s and end at v. v might not be able to get back to s.

- **Undirected:** paths go both ways, so this finds the **connected component** of s: the largest set of vertices with a path between every pair.

The adjacency list only tells you about **single edges**. A path can use many edges, so finding them takes work.

7.3.1 The idea

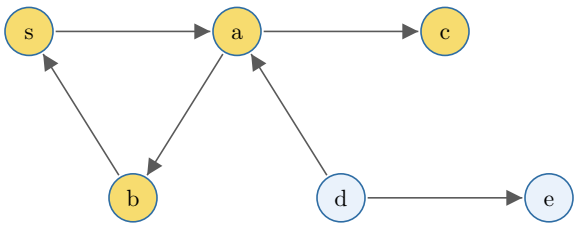
Depth-first search (DFS) and breadth-first search (BFS) both solve reachability. They do the **same important things**, and differ only in an implementation detail. So describe one general algorithm, **whatever-first search**:

1. s can reach itself (a path with no edges).
2. If s can reach u and $u \to v$ is an edge, then s can reach v. So every time you find a reachable vertex, remember to search out of it later.
3. A cycle could send the search around forever. So **mark** a vertex when you explore its edges, and never explore it again.

A **bag** is any data structure that supports **insert a vertex** and **remove some vertex** that was inserted. Which vertex comes out depends on the bag.

```
WhateverFirstSearch(s):
  put s into the bag
  while the bag is not empty
    take u from the bag
    if u is unmarked
      mark u
      for each edge u → v
        put v into the bag
```

Example: A directed graph. Searching from s marks the yellow vertices. Nothing leads from them to d or e ($d \to a$ points the wrong way):



Claim: Whatever-first search from s marks **exactly** the vertices s can reach. The full proof is in Erickson's book. You won't be asked for proofs like this. Both halves use induction:

- **Marked ⇒ reachable:** Induction on the order of marking. Either $v = s$, or v went into the bag through an edge $u \to v$ from a vertex u marked earlier. By induction u is reachable, so v is too: add one edge.

- **Reachable ⇒ marked:** Induction on the distance from s to v. If $v = s$, it goes into the bag first and comes out marked. Otherwise a shortest path ends with an edge $u \to v$, where u is closer to s. By induction u gets marked. Then v goes into the bag, and it gets marked later.

7.3.2 Running time

Let T be the time for one bag insertion or removal. Count each line:

Step	Times	Why
`mark u`	$\leq \lvert V \rvert$	Only unmarked vertices get marked, and marks never go away
`for each edge u → v`	$\leq \lvert E \rvert$ total	Only right after marking u, which happens once
`put v into the bag`	$\leq \lvert E \rvert + 1$	Once for s, then once per edge seen
`take u from the bag`	$\leq \lvert E \rvert + 1$	Can't remove more than was inserted

$$\text{Total: } O(|V| + |E| \cdot T)$$

For reachability, use the simplest bag: a **stack** or a **queue**, with $T = O(1)$:

$$\boxed{O(|V| + |E|)}$$

That is as fast as possible: the time it takes just to read the graph.

Bag	Search
Stack	Depth-first search (roughly)
Queue	Breadth-first search: also finds shortest paths, counted in edges
Priority queue	Other algorithms, with a slower T (later)

Different bags find the vertices in different orders and build different **spanning trees**. More on Thursday.

7.4 Reductions

Reducing problem A to problem B: solve an instance of A by building one or more instances of B and running an algorithm for B. From now on: build a graph, then run a known graph algorithm.

7.4.1 Example: useless DFA states

Input: a DFA $M = (Q, \{0, 1\}, \delta, s, A)$ with n states. A state q is **useless** if no input string reaches it: there is no x with $\delta^*(s, x) = q$. **Goal:** find every useless state, so you can delete them. (The subset construction for NFA → DFA often makes many of them.)

Moving from state to state along transitions is like walking from vertex to vertex. So:

1. Build a graph: $G = (V, E)$ with

$$V = Q, \qquad E = \{p \to q \mid \delta(p, 0) = q \text{ or } \delta(p, 1) = q\}$$

2. Say what graph problem to solve, and why it helps:

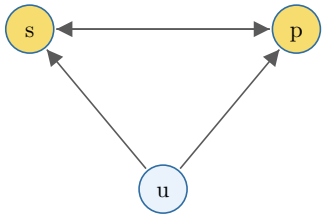
$$q \text{ is useless} \iff s \text{ cannot reach } q \text{ in } G$$

3. Say which algorithm to run, and how to use its output: Run whatever-first search from s. The useless states are the vertices it **doesn't** mark.

4. Running time, in terms of the original input: G has $|V| = n$ vertices and $|E| \leq 2n$ edges (two transitions per state). Whatever-first search takes $O(|V| + |E|)$, so the algorithm takes

$$\boxed{O(n)}$$

q	$\delta(q, 0)$	$\delta(q, 1)$
s	p	s
p	s	p
u	s	p



A small DFA and its graph (self-loops not drawn). Search from s marks s and p, so u is useless.

Key idea **That is the whole answer:** For a reduction, write down: the graph (its vertices and edges, and what they mean if it isn't obvious), the graph problem and why its answer helps, the algorithm and how to use its output, and the running time. This is exactly the detail a full-credit exam solution needs. No induction proof is needed at this point in the course.

Watch out **Say "whatever-first search", not "DFS" or "BFS":** Unless you say otherwise, it means the stack or queue version. Naming a specific search is more detail than needed, and suggests you aren't thinking about the actual problem (reachability, not shortest paths). It can lose a few points.

Watch out **Report the running time in the input's own terms:** $|V|$ and $|E|$ belong to a graph you made up. The person asking about useless states has never heard of it, so give $O(n)$, not $O(|V| + |E|)$. Building the graph is assumed to take $O(|V| + |E|)$ time. If building it is complicated, count that too.

Next time: more about the bag and spanning trees, and a reduction that builds a **new** graph out of a given graph (it's in the lecture notes on the website already, useful for the lab).

8 Lecture 17 (Oct 22) - Graph Layering, Whatever-First Search Variants, Depth-First Search

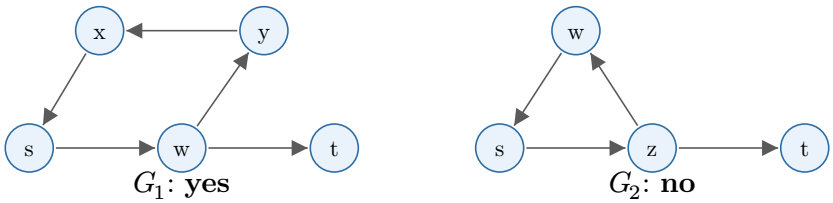
Today: one more reduction to reachability, with a technique called **graph layering**. Then what happens when the bag in whatever-first search matters, and a closer look at **depth-first search**.

8.1 Graph Layering

8.1.1 The problem

Input: a directed graph $G = (V, E)$ and two vertices s and t. **Question:** is there a **walk** from s to t whose length is divisible by 3?

A **walk** follows the edge directions, and may repeat vertices and edges. (A path may not.)



G_1: **yes** G_2: **no**

- G_1: the shortest path $s \to w \to t$ has length 2. But the walk $s \to w \to y \to x \to s \to w \to t$ has length 6. Every walk from s to t has length $2 + 4k$, and $k = 1$ works.
- G_2: there are infinitely many walks from s to t, such as $s \to z \to t$ and $s \to z \to w \to s \to z \to t$. But every one has length $2 + 3k$, which is never divisible by 3.

8.1.2 Don't modify the search, build a new graph

It's tempting to change the code of whatever-first search so it also tracks lengths mod 3. That can work, but it's **dangerous**. It's easy to get wrong, and you would have to prove that your modified search still finds every walk. Instead, **reduce to plain reachability** in a new graph G', and run the search unchanged.

Imagine an ant walking on G. To answer the question, it only needs to know two things: **where it is**, and **how many steps it has taken, mod 3**. Make each pair a vertex.

1. Build a graph: $G' = (V', E')$ with

$$V' = V \times \{0, 1, 2\}$$
$$E' = \{(u, i) \to (v, (i + 1) \bmod 3) \mid u \to v \in E, i \in \{0, 1, 2\}\}$$

Each edge of G becomes 3 edges of G', one **from each layer to the next**. Following an edge always adds one step. The arrow notation keeps this readable: an edge between pairs written with nested pairs gets ugly fast.

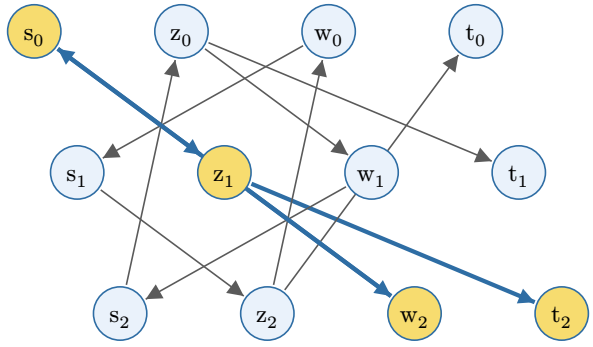
2. Graph problem, and why it helps:

G has a walk from s to t of length divisible by $3 \iff (s, 0)$ can reach $(t, 0)$ in G'

3. Algorithm: Run whatever-first search in G' from $(s, 0)$. Answer yes if and only if it marks $(t, 0)$.

4. Running time: $|V'| = 3|V|$ and $|E'| = 3|E|$, so building G' and searching it takes $O(|V'| + |E'|) = \boxed{O(|V| + |E|)}$. That's as fast as possible.

G_2, layered: Layer i holds the vertices v_i (meaning "at v, after i steps mod 3"). The search from s_0 marks the yellow vertices and follows the bold edges. It never marks t_0, so the answer is no:



Layers 0, 1, 2 from top to bottom. Every edge goes down one layer, except edges out of layer 2, which wrap around to layer 0.

For G_1, the walk of length 6 becomes the path $s_0 \to w_1 \to y_2 \to x_0 \to s_1 \to w_2 \to t_0$ in G'.

> **Key idea** **Graph layering:** when a graph problem (reachability, shortest paths) needs to remember **extra information**, make one copy of the graph per value of that information, and wire the copies together so the layer you're in **is** the information. Same idea as the product construction for DFAs: new states = old states × extra bits. Erickson has a separate note on graph layering, linked from the course schedule.

The new graph is an ordinary graph, so you can hand it to **any** black-box graph algorithm (whatever-first search, Dijkstra's algorithm, …). The algorithm doesn't care where the vertices came from.

8.2 Whatever-First Search, Revisited

8.2.1 Parent pointers

Put **pairs** into the bag instead of single vertices. A pair (p, v) means "found v, and the last step to it came from p":

```
WhateverFirstSearch(s):
  put (ø, s) into the bag
  while the bag is not empty
    take (p, v) from the bag
    if v is unmarked
```

```
mark v
v.parent ← p
for each edge v → w
  put (v, w) into the bag
```

Every marked vertex except s gets exactly one **parent edge** $v.\text{parent} \to v$, and its parent was marked earlier. So the parent edges form a **tree rooted at s**: it holds one path from s to every vertex s can reach. In an undirected graph, this is a **spanning tree** of s's component: a tree containing every vertex of it.

8.2.2 The bag matters now

A different bag removes vertices in a different order. That gives a **different tree**, and a different running time $O(|V| + |E| \cdot T)$, where T is the time per bag operation:

Bag	**Search, and the tree it builds**	T	**Total time**
Queue	**Breadth-first search**. Finds every vertex 1 step from s, then 2 steps, and so on. Builds a **shortest-path tree**, counting edges (no weights).	$O(1)$	$O(\|V\| + \|E\|)$
Priority queue: priority of (u, v) is $\text{dist}(u) + w(u \to v)$; weights ≥ 0	**Best-first search = Dijkstra's algorithm**. Shortest paths with edge weights (proof next week).	$O(\log\|E\|)$	$O(\|V\| + \|E\| \log\|V\|)$
Priority queue: priority of uv is $w(uv)$; undirected	**Prim's algorithm**. Builds a **minimum spanning tree** (not covered this semester).	$O(\log\|E\|)$	$O(\|V\| + \|E\| \log\|V\|)$
Stack	**Depth-first search**. Long, skinny trees. Uses: below.	$O(1)$	$O(\|V\| + \|E\|)$

Here $\text{dist}(u)$ is the total weight of the tree path from s to u. The priority queue is a binary heap, with $O(\log|E|)$ insert and extract-min. A simple graph has $|E| \leq |V|^2$, so $\log|E| \leq 2\log|V|$, and $O(\log|E|) = O(\log|V|)$.

> **Key idea** **BFS, DFS, Dijkstra and Prim are all whatever-first search:** They differ only in the bag that decides where to go next.

On homework 8 you may use Dijkstra's algorithm as a black box.

8.3 Depth-First Search

For BFS, Dijkstra and Prim there is an obvious problem to solve. For depth-first search there isn't a one-sentence reason to care. So this time, start with the algorithm and then look for problems it solves.

8.3.1 The recursive version

Whatever-first search with a stack is a depth-first search, but DFS is usually written **recursively**: for each edge $v \to w$, immediately explore everything out of w. One more change: check whether w is marked **before** the recursive call, instead of pushing it and checking later.

```
DFS(v):                        ⟨⟨v is unmarked⟩⟩
  mark v
  for each edge v → w
    if w is unmarked
      w.parent ← v
      DFS(w)
```

Like any search, DFS(s) marks everything reachable from s (if nothing was marked before). What that is depends heavily on s. On a path, starting at the first vertex marks everything, and starting at the last vertex marks only that vertex.

The uses of DFS are about the structure of the **whole** graph, so call it until every vertex is marked:

```
DFSAll(G):
  for all vertices v
    unmark v
  for all vertices v
    if v is unmarked
      DFS(v)
```

Every vertex is marked once and its edges are explored once: $O(|V| + |E|)$ time.

8.3.2 Three facts about DFSAll

1. **Every vertex gets marked**: DFS(v) is called exactly once for every v. The loop in DFSAll guarantees it.
2. **Before DFS(v) finishes, it marks every vertex reachable from v through unmarked vertices.** It keeps diving deeper. It stops only at vertices that someone else already marked.
3. **If DFS(v) calls DFS(w), directly or indirectly, there is a path from v to w.**

8.3.3 Clock: start and finish times

The applications depend on the **order** in which DFS starts and finishes each vertex. To reason about it, add a clock. Each event gets the next clock value:

```
DFSAll(G):
  clock ← 0
  for all vertices v
```

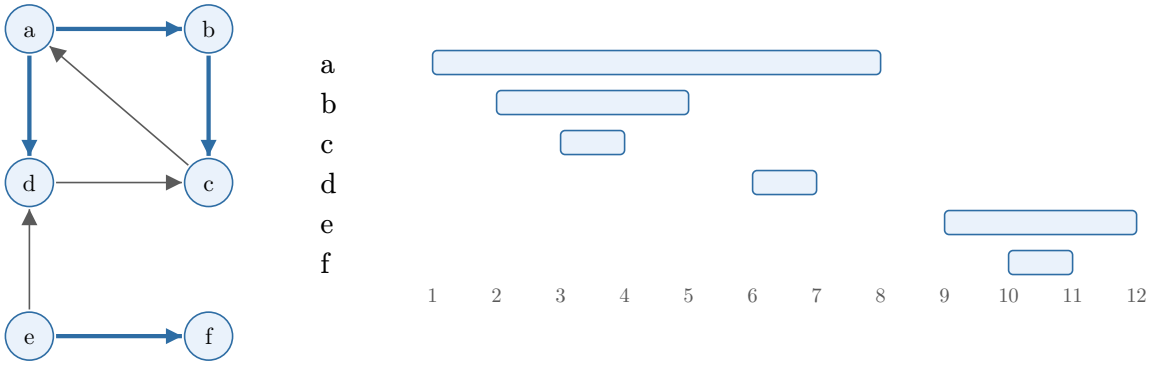
```
    unmark v
  for all vertices v
    if v is unmarked
      clock ← DFS(v, clock)

DFS(v, clock):
  mark v
  clock ← clock + 1
  v.pre ← clock                ⟨⟨start time⟩⟩
  for each edge v → w
    if w is unmarked
      w.parent ← v
      clock ← DFS(w, clock)
  clock ← clock + 1
  v.post ← clock               ⟨⟨finish time⟩⟩
  return clock
```

The **active interval** of v is $[v.\text{pre}, v.\text{post}]$: from when DFS($v$) goes onto the call stack until it comes off.

> **Key idea** **Active intervals are either disjoint or nested:** That is how a call stack works: if DFS(u) starts while DFS(v) is running, DFS(u) finishes first.

Example: DFSAll on this graph, taking vertices and edges in alphabetical order. Bold edges are tree (parent) edges:



DFS(a) finds b, then c (whose edge to a goes to a marked vertex), then backs up to a and finds d ($d \to c$: already marked). Nothing marked so far reaches e, so DFSAll calls DFS(e) later, which finds f.

Order	**Sort by**
Preorder: a, b, c, d, e, f	start time v.pre. Rarely useful in this class.
Postorder: c, b, d, a, f, e	finish time v.post. Very useful.

8.4 Finding a Cycle

A **dependency graph** has the subproblems of a recursive algorithm as vertices, and an edge $u \to v$ when u calls v directly (Fibonacci: $F_8 \to F_7$ and $F_8 \to F_6$). A recursive algorithm **terminates only if its dependency graph has no cycle**. On a cycle through u, solving u needs the answer to u, which you're still computing.

Problem: given a directed graph G, does it have a cycle? A directed graph with no cycle is a **directed acyclic graph** (DAG).

Run DFSAll. Suppose that during DFS(v) there is a call to DFS(u), and it finds an edge $u \to v$ back to the still-active v. That edge is a **back edge**. By fact 3 there is a path $v \rightsquigarrow u$, and with the edge $u \to v$ that makes a **cycle**. In the example, $c \to a$ is a back edge: cycle $a \to b \to c \to a$.

> **Key idea** **One back edge proves a cycle:** Which edges are back edges depends on the order DFSAll takes vertices and edges. But if DFSAll finds even one, the graph is not acyclic.

Is the converse true (a cycle $\Rightarrow$ DFS always finds a back edge)? Next lecture.

9 Lecture 18 (Oct 27) - Cycle Detection, Topological Sort, Shortest Paths, Dijkstra's Algorithm

First, finish depth-first search: detect cycles, and get a topological sort almost for free. Then start the last big problem of the algorithms part of the course: **shortest paths**.

9.1 Recap: DFS with a Clock

From Lecture 17: DFSAll unmarks every vertex, then calls DFS(v) on each vertex that is still unmarked. DFS(v) marks v, records its start time v.pre, recursively explores every unmarked out-neighbor, and records its finish time v.post. Two facts used today:

- If there is a path from v to w through vertices that are unmarked when DFS(v) starts, then DFS(v) calls DFS(w) (maybe indirectly). It may not follow that exact path.
- Sorting vertices by v.post gives **a postorder**. Different loop orders give different postorders.

An edge $u \to v$ is a **back edge** if DFS(v) (directly or indirectly) calls DFS(u). Then there is a path $v \rightsquigarrow u$, and with $u \to v$ it is a cycle.

9.2 Cycle Detection

Theorem 1. *G has a directed cycle if and only if DFSAll on G finds a back edge.*

($\Longleftarrow$): a back edge closes a cycle (Lecture 17).

($\Longrightarrow$): let C be a cycle. DFSAll calls DFS on every vertex, so let v be the **first** vertex of C that DFS is called on. Let u be the vertex just before v on C, so $u \to v$ is an edge. When DFS(v) starts, the rest of C is still unmarked, so the path around C from v to u uses only unmarked vertices. So DFS(v) calls DFS(u), and the edge $u \to v$ is a back edge.

Every cycle has **at least one** edge found as a back edge. Which one depends on the loop orders in DFSAll and DFS.

9.2.1 Spotting back edges with finish times

If $u \to v$ is a back edge, DFS(u) runs inside DFS(v):

$$v.\text{pre} < u.\text{pre} < u.\text{post} < v.\text{post}$$

You only need to compare finish times. For **any** edge $u \to v$, two other orders are impossible:

- Active intervals never partially overlap, because DFS uses a call stack.
- u.post $< v$.pre can't happen: DFS(u) sees the edge $u \to v$, so it marks v before it finishes.

So for an edge $u \to v$:

$$u \to v \text{ is a back edge} \iff u.\text{post} < v.\text{post}$$

```
IsAcyclic(G):
  DFSAll(G)                          ⟨⟨with the clock⟩⟩
  for each edge u → v
    if u.post < v.post
      return False                   ⟨⟨back edge: cycle⟩⟩
  return True
```

Running time: DFSAll marks each vertex once and looks at each edge once, and the loop is $O(1)$ per edge: $\boxed{O(|V| + |E|)}$.

Erickson's book does it another way: store a status on each vertex (new, active, finished), and report a back edge when you find an edge into an **active** vertex.

9.3 Topological Sort

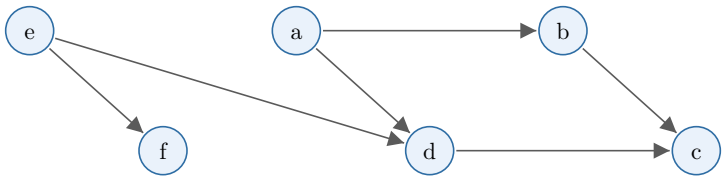
A recursive algorithm can only work if its dependency graph has no cycle. If it has none, you still need an order to solve the subproblems in: every subproblem before anything that points to it.

A **topological order** of a directed graph is an ordering of its vertices such that for every edge $u \to v$, $\boldsymbol{u}$ **comes before** $\boldsymbol{v}$. A dependency graph needs the reverse (callees first), which is a little annoying.

- **A cycle means no topological order:** Going around the cycle, some edge has to point backward.
- **No cycle: reverse a postorder:** With no cycle, DFSAll finds no back edge. So every edge $u \to v$ has v.post $< u$.post: the head finishes first. List the vertices by **decreasing** finish time, and every edge points forward.

> **Key idea** **Topological order = reversed postorder:** Run DFSAll, then list the vertices from last finished to first finished. $O(|V| + |E|)$. The same run also checks for a cycle.

Example: The Lecture 17 graph without the edge $c \to a$, so it is a DAG. DFSAll in alphabetical order gives postorder c, b, d, a, f, e. Reversed:



Topological order e, f, a, d, b, c, left to right. Every edge points right.

> **Watch out** **This is not the other topological sort algorithm** (repeatedly remove a vertex with no incoming edges). Both are correct. The DFS one goes with cycle detection and with dynamic programming on dependency graphs (Thursday).

9.4 Single-Source Shortest Paths

Input: a directed graph $G = (V, E)$, a weight function $w : E \to \mathbb{R}$, and a source vertex s. Weights may be positive, zero or even negative. Think of a road network where $w(u \to v)$ is the travel time on a stretch of road.

The **distance** $\text{dist}(s, t)$ is the minimum total weight of any path from s to t. Every known algorithm that finds one distance finds the distances from s to **every** vertex anyway: the shortest route to t needs the shortest route to the vertex before t, and so on. So the problem is **single-source shortest paths** (SSSP).

Negative weights still make sense (a scenic road you'd like to take, or the lab and homework this week). Today all weights are **positive**. Negative weights are on Thursday.

9.4.1 The shortest-path tree

A subpath of a shortest path is a shortest path: If the part from a to d weren't the shortest from a to d, replacing it with a shorter one would shorten the whole path.

So if two shortest paths split apart at a and meet again at d, both detours from a to d are shortest. Keep one of them for both paths ("uncross" them). Doing this again and again leaves a **shortest-path tree rooted at s**.

Goal: compute $\text{dist}(s, v)$ for every v, and a shortest-path tree rooted at s.

9.5 Ford's Generic Algorithm

Like whatever-first search for reachability, one general framework covers all the shortest-path algorithms. Each specific algorithm chooses an order that suits the graph's structure and weights. The framework is Lester Ford's (1956). Each vertex stores:

- v.dist: the length of **some** path from s to v, or ∞. It can be too big until the algorithm finishes.
- v.pred: the vertex before v on that path, or Null.

Following $v, v.\text{pred}, v.\text{pred.pred}, \ldots$ back to s gives a path. But it may not have length v.dist, because earlier pred values can change later. What's guaranteed: **some** path ending in $v.\text{pred} \to v$ has length v.dist.

Tense edge: $u \to v$ is **tense** if

$$u.\text{dist} + w(u \to v) < v.\text{dist}$$

That is proof you're not done: going to u and then along $u \to v$ beats the best known path to v. Fix it by **relaxing** the edge.

```
InitSSSP(s):
  s.dist ← 0
  s.pred ← Null
  for every vertex v ≠ s
```

```
    v.dist ← ∞
    v.pred ← Null

Relax(u → v):
  v.dist ← u.dist + w(u → v)
  v.pred ← u

FordSSSP(s):
  InitSSSP(s)
  while there is at least one tense edge
    Relax any tense edge
```

Claim (positive weights): FordSSSP terminates. When it does, every v.dist $= \text{dist}(s, v)$, and every v.pred is the vertex before v on a shortest path, so the pred edges form a shortest-path tree. Proof idea:

- While some v.dist is too big, some edge is tense, so it can't stop early.
- Each v.dist is always the length of a real path, and it only goes down. There are only finitely many paths, so it can't go down forever.

See Erickson's shortest-paths chapter for more detail.

> **Watch out** **Don't just say "run Ford's algorithm":** With a bad order of tense edges it can take **exponential** time: one v.dist might drop about $2^{|V|}$ times. Graders assume the worst order. Look at the graph's structure (cycles? a DAG?) and its weights (all positive? all 1? some negative?), and pick the specific algorithm that fits.

9.6 Dijkstra's Algorithm

For **positive** weights, nothing else assumed. Two observations about FordSSSP:

1. **An edge becomes tense only when its tail's dist drops:** Dist values only decrease. If $u \to v$ isn't tense, lowering v.dist can't make it tense. Only lowering u.dist can. So, like whatever-first search, revisit u's outgoing edges after u.dist drops, and not before.
2. **Relax out of the vertex with the smallest dist:** With positive weights, relaxing $u \to v$ gives v.dist $>$ u.dist. So the smallest dist value can never go down again.

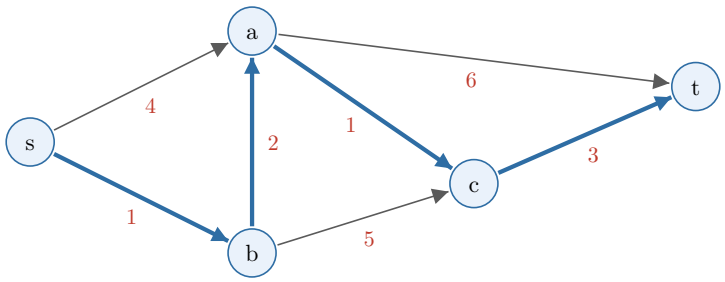
Together: keep the vertices whose dist dropped in a **priority queue**, keyed by dist.

```
Dijkstra(s):
  InitSSSP(s)
  Insert(s, 0)
  while the priority queue is not empty
    u ← ExtractMin()
    for each edge u → v
      if u → v is tense
        Relax(u → v)
```

```
if v is in the priority queue
  DecreaseKey(v, v.dist)
else
  Insert(v, v.dist)
```

The priority queue starts with only s, not every vertex. A vertex is inserted when its dist first drops. As written, a vertex could even be inserted **again** after it was extracted. With positive weights it never is (see below). Why it is written this way: Thursday.

Example: Edge weights in red. Bold edges: the final pred edges, which form the shortest-path tree:



After	s	a	b	c	t
InitSSSP	0	∞	∞	∞	∞
extract s	**0**	4	1	∞	∞
extract b		3 (via b)	**1**	6	∞
extract a		**3**		4 (via a)	9
extract c				**4**	7 (via c)
extract t					**7**

Bold: the vertex is extracted and its dist is final. The extracted dist values come out in increasing order: $0, 1, 3, 4, 7$.

9.6.1 Running time

Let d_i be the dist of the ith vertex extracted. Every new dist value created after that is d_i plus a **positive** weight, so it is bigger than d_i. Since d_i was the smallest in the queue, the extracted values **strictly increase**: $d_1 < d_2 < ...$

A vertex's dist never goes up. So if v were extracted twice, its second value would be no bigger than its first, contradicting the increasing order. **Each vertex is extracted at most once.** So:

- at most $|V|$ Insert and $|V|$ ExtractMin operations;
- each edge is examined at most once (when its tail is extracted), so at most $|E|$ DecreaseKey operations.

With a binary heap, each operation takes $O(\log|V|)$:

$$O((|V| + |E|)\log|V|) = \boxed{O(|E|\log|V|)}$$

The last step assumes every vertex is reachable from s, so $|E| \geq |V| - 1$. A simple graph has $|E| \leq |V|^2$, so $\log|E| \leq 2\log|V|$: $\log|E|$ and $\log|V|$ are the same in big-O.

Appendix A: Final Review: Patterns and Connections

One place to review for the final. The lecture numbers point back to the full notes.

A.1 Which Technique?

The most important exam skill: look at a new problem and recognize what it needs.

If the problem...	**Try...**	**See**
Asks to prove a fact about strings or another recursively defined object	Strong induction on the structure: $w = \varepsilon$ or $w = ax$	Lec 01, Lab 1A
Asks to describe a language by a pattern	Regular expression	Lab 1B
Contains smaller copies of itself	Recursion: shrink, trust, combine	Lec 10
Is easy to finish once both halves are solved	Divide and conquer (mergesort)	Lec 10
Asks for the kth smallest element or the median	Selection: quickselect, or median of medians for $O(n)$ worst case	Lec 11
Does arithmetic on huge n-digit numbers	Split the digits into halves, and make as few recursive calls as possible	Lec 12
Needs a sequence of decisions (list all / find best / is it possible?)	Backtracking: one decision, try every option, recurse	Lec 12
Has a sequence or string input, and you choose the next piece	Backtracking with the subproblem "the suffix starting at index i"	Lec 12, 13
Has a backtracking recurrence whose calls repeat	Dynamic programming	Lec 13
Splits a sequence into two independent parts, and you choose where	Interval DP: subproblem = subarray $A[i..k]$; try every split point	Lec 15
Has a rooted tree as input	Tree DP: one subproblem per subtree, named by its root; evaluate in postorder	Lec 15
Is NP-hard on general graphs (such as max independent set), but the input is a tree	Tree DP: decide the root (in or out), recurse on subtrees	Lec 15

If the problem...	Try...	See
Asks "can I get from here to there?" (states, configurations, moves)	Build a graph whose vertices are the objects; reachability with whatever-first search	Lec 16
Is reachability or shortest paths, but you must remember extra information (steps mod k, a parity, a flag)	Graph layering: vertices $V \times$ {extra info}; search or Dijkstra on the new graph	Lec 17
Asks for shortest paths counted in edges	Breadth-first search (whatever-first search with a queue)	Lec 17
Asks for shortest paths with weights ≥ 0	Dijkstra's algorithm, as a black box	Lec 17, 18
Asks whether a directed graph has a cycle (or whether a recursion terminates)	DFSAll; a back edge means a cycle. Check every edge for $u.\text{post} < v.\text{post}$	Lec 17, 18
Asks for an order where every task comes after the tasks it needs (a DAG)	Topological sort: DFSAll, then reverse the postorder	Lec 18
Asks for distances or shortest paths from one vertex	Single-source shortest paths: pick the algorithm that fits the structure and the weights	Lec 18

Key idea **If something from class works as a black box, use it:** Sorting and then picking the middle element is a correct $O(n \log n)$ median algorithm.

A.2 Algorithms at a Glance

Algorithm	Idea	Recurrence ⇒ worst-case time
Tower of Hanoi	Move $n-1$ disks off, move the big disk, move $n-1$ back	$T(n) = 2T(n-1) + 1$ $\Rightarrow 2^n - 1$ moves
Mergesort	Sort both halves, then merge	$T(n) = 2T(n/2) + O(n)$ $\Rightarrow O(n \log n)$
Quicksort	Partition around a pivot, sort both sides	$T(n) = T(n-1) + O(n)$ $\Rightarrow O(n^2)$

Algorithm	**Idea**	**Recurrence ⇒ worst-case time**
Quickselect	One-armed quicksort: recurse only on the side holding rank k	$T(n) = T(n-1) + O(n)$ ⇒ $O(n^2)$
Median of medians	Pivot = median of the block-of-5 medians, found recursively	$T(n) \le T(n/5) + T(7n/10) + O(n)$ ⇒ $O(n)$
Split multiply	Split digits in half; 4 products ac, ad, bc, bd	$T(n) = 4T(n/2) + O(n)$ ⇒ $O(n^2)$
Karatsuba	3 products: $ad + bc = ac + bd - (a - b)(c - d)$	$T(n) = 3T(n/2) + O(n)$ ⇒ $O(n^{\log_2 3}) \approx O(n^{1.585})$
RecFibo	Straight from the definition; $\varphi \approx 1.618$	$T(n) = T(n-1) + T(n-2) + 1$ ⇒ $\Theta(\varphi^n)$
IterFibo	Fill $F[0..n]$ left to right	one loop ⇒ $O(n)$
Text segmentation	Try every first word, recurse on the suffix	backtracking ⇒ $O(2^n)$ IsWord calls
FastSplittable	Fill SplitTable$[1..n+1]$ right to left	two nested loops ⇒ $O(n^2)$ IsWord calls
Woodcutting	Try every first cut r; fill Cutting$[i, k]$ with i down, k up	$O(n^2)$ entries × $O(n)$ ⇒ $O(n^3)$
Tree MIS	Root in (skip children) or out (children free); postorder	charge reads to nodes ⇒ $O(n)$
Whatever-first search	Bag of vertices; mark on removal; push all out-neighbors	$O(\|V\| + \|E\| \cdot T)$ ⇒ $O(\|V\| + \|E\|)$ with a stack or queue

Algorithm	**Idea**	**Recurrence ⇒ worst-case time**
Useless DFA states	Graph on states, edge per transition; unmarked by search from s	$\|V\| = n, \|E\| \leq 2n$ $\Rightarrow O(n)$
Breadth-first search	Whatever-first search with a queue; shortest-path tree (edge count)	$T = O(1)$ $\Rightarrow O(\|V\| + \|E\|)$
Dijkstra	Extract the min-dist vertex, relax its tense out-edges; weights > 0	each vertex extracted once, binary heap $\Rightarrow O(\|E\| \log \|V\|)$
DFS / DFSAll	Recursive; check marks before calling; pre/post clock times	each vertex and edge once $\Rightarrow O(\|V\| + \|E\|)$
IsAcyclic	DFSAll; a cycle exists iff some edge has $u.\text{post} < v.\text{post}$	DFSAll + one pass over edges $\Rightarrow O(\|V\| + \|E\|)$
Topological sort	DFSAll; list vertices by decreasing $v.\text{post}$	DFSAll $\Rightarrow O(\|V\| + \|E\|)$
FordSSSP	Relax any tense edge until none is left (generic)	bad order $\Rightarrow$ exponential
Walk of length $\equiv 0 \pmod 3$	Layered graph $V \times \{0, 1, 2\}$; can $(s, 0)$ reach $(t, 0)$?	$\|V'\| = 3\|V\|$, $\|E'\| = 3\|E\|$ $\Rightarrow O(\|V\| + \|E\|)$

A.3 Solving Recurrences: Recursion Trees

1. Draw one node per call. A node's value is the work in that call, **not** counting its children.
2. Sum each level, then look at how the level sums change from top to bottom.

Level sums	Total	Example
All equal	level sum × number of levels	Mergesort: $n \cdot \log_2 n$
Decreasing geometric	$O(\text{root})$	Median of medians: $O(n)$
Increasing geometric	$O(\#\text{ leaves})$	Karatsuba: $O\left(n^{\log_2 3}\right)$
Arithmetic (not geometric)	largest term × number of terms	Quicksort worst: $O(n^2)$

Useful facts:

- Number of leaves = (# calls per node)^{depth}.
- Size multiplied by $r < 1$ each level $\Rightarrow$ depth $= \log_{1/r} n = O(\log n)$.
- $a^{\log_b c} = c^{\log_b a}$, for example $4^{\log_2 n} = n^{\log_2 4} = n^2$.
- The log base, floors and ceilings don't change the big-O answer.
- Unbalanced splits are fine: a tree with missing nodes still gives an upper bound.

Watch out Don't rely on a formula theorem. It only fits equal-size pieces, and median of medians doesn't fit. Recursion trees always work.

A.4 Recipes

A.4.1 Induction proof (Lec 01)

1. State a **strong** inductive hypothesis explicitly ("for all $k < n$").
2. Handle **every case** of the object (for strings: $w = \varepsilon$ and $w = ax$).
3. Use the hypothesis where it helps.
4. End by noting that all cases are handled.

A.4.2 Describing an algorithm (Lec 10)

- English or pseudocode, never real code.
- Say what every function **returns**.
- Running time: give the recurrence **and** its solution.

A.4.3 Backtracking (Lec 12)

1. See the problem as a **sequence of decisions**.
2. Find the **recursive subproblem**: what the recursion fairy needs to know to finish.
3. Try **every option** for one decision, and recurse on each.
4. Pick a **base case** where no decisions are left.
5. Write it as a **recurrence** on indices, and say which call solves the problem.

A.4.4 Dynamic programming (Lec 13)

Step	Write down
1a. Subproblem	English definition of what the function returns. Not how to compute it, not how it gets used.
1b. Recurrence	Including base cases.
1c. Original problem	Which call answers it, for example Splittable(1).
2a. All subproblems	The range of every parameter, for example $1 \leq i \leq n+1$.
2b. Data structure	Usually an array, named after the function.
2c. Dependencies	Which entries each entry reads. Draw them.
2d. Evaluation order	An order where every dependency is filled first.
2e. Algorithm	Loops around the recurrence (optional on exams).
2f. Running time	(Number of subproblems) $\times$ (time per subproblem).

A.4.5 DP on a tree (Lec 15)

Same recipe, with these changes:

- **Subproblems** are named by nodes: "the subtree rooted at v".
- **Data structure:** a field on each node (v.MIS), not an array.
- **Evaluation order:** postorder. Saying "postorder" is a complete answer.
- **Running time:** add the work over all nodes and charge each read to the child (or edge) being read. Don't multiply the maximum time per node by n.

A.4.6 Graph reduction (Lec 16)

Step	Write down
1. Graph	Vertices and edges (and what they mean, if not obvious).
2. Problem	Which graph problem, and why its answer solves yours (q useless $\iff$ s can't reach q).
3. Algorithm	Which algorithm, and how to use its output ("whatever-first search from s; the unmarked vertices").
4. Running time	In terms of the **original input** (n), not $\lvert V\rvert$ and $\lvert E\rvert$.

A.4.7 Graph layering (Lec 17)

1. Ask what the "ant" walking on G must remember besides where it is (steps mod 3, a parity, whether it used something). Call that set S.
2. $V' = V \times S$. For each edge $u \to v$ and each $i \in S$, add the edge $(u, i) \to (v, i')$, where i' is the new extra information after that step.
3. Ask a plain graph question: can (s, start) reach (t, goal)? Shortest path?

4. Running time: the new graph has $|S|$ times as many vertices and edges. Give it in terms of the original input.

A.4.8 Shortest paths (Lec 18)

- Each vertex keeps v.dist (length of **some** path, or ∞) and v.pred.
- **Tense:** $u.\text{dist} + w(u \to v) < v.\text{dist}$. **Relax:** set $v.\text{dist} \leftarrow u.\text{dist} + w(u \to v)$ and $v.\text{pred} \leftarrow u$.
- Every algorithm is Ford's: relax tense edges until none are left. They differ in the order. Pick the order from the graph's structure and weights.

A.4.9 Graph representations (Lec 16)

Operation	Adjacency matrix	Adjacency list (default)
Is uv an edge?	$O(1)$	$O(\deg(u))$
Neighbors of u	$O(\|V\|)$	$O(1)$ each
Space	$O(\|V\|^2)$	$O(\|V\| + \|E\|)$

A.5 Connections

- **Recursion fairy = inductive hypothesis:** Trusting a recursive call on a smaller input is the same move as using the hypothesis in a strong induction proof.
- **Generalize the problem to recurse on it:** Median → selection of rank k (Lec 11). Segment the string → segment the suffix $A[i..n]$ (Lec 12). Same idea as strengthening a hypothesis in induction.
- **Quicksort and quickselect** share `Partition`. Quickselect recurses on one side only, so its good-pivot tree shrinks instead of staying flat.
- **The number of recursive calls matters:** Karatsuba's 3 calls instead of 4: $n^2 \to n^{1.585}$. Median of medians with blocks of 5 instead of 3: $n \log n \to n$.
- **Subproblems can be unexpected:** Median of medians recurses on a sample just to find a pivot.
- **DP = backtracking + memoization, then iteration:** Memoized Fibonacci fills its array left to right; IterFibo makes that order explicit. Same for FastSplittable, right to left.
- **A decision that splits the input into independent pieces** means you add their costs: left and right subplank (woodcutting), subtrees of the root (tree MIS), left and right halves (mergesort).
- **Woodcutting builds a binary tree:** Choosing the first cut = choosing the root. Try every root; don't try to be clever.
- **Split one subproblem into two to simplify the recurrence:** MISyes and MISno remove the grandchildren sums. Same move as generalizing a problem to recurse on it.

- **Evaluation order = dependencies first:** Right to left (FastSplittable), row by row with i down and k up (woodcutting), postorder (trees). Dependency graphs (Lec 16) make these orders explicit.
- **Vertices can be anything:** Configuration graphs (Tower of Hanoi), dependency graphs (Fibonacci calls), NFA/DFA state graphs. Reachability in them answers "can this happen?".
- **BFS, DFS, Dijkstra and Prim are the same algorithm**, whatever-first search, with a different bag: queue, stack, priority queue.
- **Graph layering = product construction:** New vertices = old vertices × extra information, just like DFA states × extra bits.
- **Postorder again:** Tree DP evaluates in postorder (Lec 15); DFS finish times give a postorder of any graph (Lec 17).
- **A recursion terminates only if its dependency graph is acyclic:** DFS finds cycles through back edges.
- **DFS active intervals nest or are disjoint**, because DFS uses a call stack. That is why a back edge is exactly an edge with $u.\text{post} < v.\text{post}$.
- **Topological order is the reverse of a DP evaluation order:** A dependency graph needs callees first: postorder. Tree DP's postorder (Lec 15) is the special case.
- **A subpath of a shortest path is a shortest path:** Same "optimal substructure" that makes dynamic programming work, and why shortest paths form a tree.
- **Dijkstra = whatever-first search with a priority queue**, plus relaxing.
- **Ford's framework is to shortest paths what whatever-first search is to reachability:** one general algorithm, many orders.
- **Same recurrence, different meaning:** RecFibo's running time satisfies the Fibonacci recurrence itself, so it grows like φ^n.

A.6 Common Ways to Lose Points

- A loose bound "just to be safe" (such as $O(n!)$) is marked wrong. Give the bound your analysis produces.
- All running times are **worst case**. Quicksort is $O(n^2)$, not $O(n \log n)$.
- A greedy algorithm without a proof is assumed wrong.
- Tracing inside a recursive call instead of trusting it.
- DP: missing the English subproblem definition, or a definition that says how to compute it.
- DP: arrays named `DP`, `OPT`, `ans` or `table`.
- DP: memoizing in a hash table instead of an array.
- Skipping steps of the DP recipe on homework and exams.

- Interval DP: both loops going the same direction. Read each direction off the dependency drawing (i down, k up for woodcutting).
- Tree DP: running time as (max time per node) $\times\, n$. Charge reads to nodes.
- Graph reductions: saying "DFS" or "BFS" when you mean "whatever-first search".
- Graph reductions: giving the running time as $O(|V| + |E|)$ instead of in terms of the original input.
- Modifying the code of a search to track extra information. Build a layered graph and run the search unchanged.
- Dijkstra with negative edge weights.
- Saying "run Ford's algorithm" without an order: graded as exponential time.
- Back-edge test backwards: a cycle needs an edge with $u.\text{post} < v.\text{post}$ (the tail finishes first).
- Topological order vs. postorder: topological order is the **reverse** of postorder.

Appendix B: Labs

B.1 Lab 1A: String Induction

The following problems ask you to prove some "obvious" claims about recursively-defined string functions. In each case, we want a self-contained, step-by-step induction proof that builds on formal defintions and prior results, *not* on intuition. In particular, your proofs must refer to the formal recursive defintions of string length and string concatenation:

$$|w| := \begin{cases} 0 & \text{if } w = \varepsilon \\ 1 + |x| & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$

$$w \bullet z := \begin{cases} z & \text{if } w = \varepsilon \\ a \cdot (x \bullet z) & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$

You may freely use the following results:

Lemma 1. *$w \bullet \varepsilon = w$ for all strings w.*

Lemma 2. *$|w \bullet z| = |w| + |z|$ for all strings w and z.*

Lemma 3. *$(w \bullet y) \bullet z = w \bullet (y \bullet z)$ for all strings w, y, and z.*

The **reversal w^R** of a string w is defined recursively as follows

$$w^R := \begin{cases} \varepsilon & \text{if } w = \varepsilon \\ x^R \bullet a & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$

For example, $\texttt{STRESSED}^R = \texttt{DESSERTS}$ and $\texttt{WTF374}^R = \texttt{473FTW}$.

1. Prove that $|w| = |w^R|$ for every string w.

   Assume that $|x| = |x^R|$ for every string x shorter than w.

   Case 1: Suppose that $w = \varepsilon$.

   Then,

   $$\begin{aligned} |w^R| &= |\varepsilon^R| \\ &= |\varepsilon| \quad \text{by definition of reversal} \\ &= |w| \end{aligned}$$

   Case 2: $w = ax$ for some symbol a and string x.

Then,

$$\begin{aligned}
|w^R| &= |x^R \bullet a| && \text{by definition of reversal} \\
&= |x^R| + |a| && \text{by definition of } \textbf{Lemma 2} \\
&= |x^R| + 1 && \text{by definition of length (since } |a| = |a\varepsilon| = 1 + |\varepsilon| = 1) \\
&= |x| + 1 && \text{by defintion of inductive hypothesis} \\
&= |ax| && \text{by defintion of length} \\
&= |w|
\end{aligned}$$

2. Prove that $(w \bullet z)^R = z^R \bullet w^R$ for all strings w and z.

   Assume that $(x \bullet z)^R = z^R \bullet x^R$ for every string x shorter than w (and every string z)

   Case 1: Suppose that $w = \varepsilon$.

   Then,

$$\begin{aligned}
(w \bullet z)^R &= (\varepsilon \bullet z)^R \\
&= z^R && \text{by definition of concatenation} \\
&= z^R \bullet \varepsilon && \text{by definition of } \textbf{Lemma 1} \\
&= z^R \bullet \varepsilon^R && \text{by definition of reversal} \\
&= z^R \bullet w^R
\end{aligned}$$

   Case 2: Suppose that $w = ax$ for some symbol a and string x.

   Then,

$$\begin{aligned}
(w \bullet z)^R &= (ax \bullet z)^R \\
&= (a \cdot (x \bullet z))^R && \text{by definition of concatenation} \\
&= (x \bullet z)^R \bullet a && \text{by definition of reversal} \\
&= (z^R \bullet x^R) \bullet a && \text{by definition of inductive hypothesis} \\
&= z^R \bullet (x^R \bullet a) && \text{by definition of } \textbf{Lemma 3} \\
&= z^R \bullet w^R && \text{by definition of reversal}
\end{aligned}$$

3. Prove that $\left(w^R\right)^R = w$ for every string w

   Assume that $\left(x^R\right)^R = x$ for every string x shorter than w.

   Case 1: Suppose that $w = \varepsilon$.

   Then,

$$
\begin{aligned}
\left(w^R\right)^R &= \left(\varepsilon^R\right)^R \\
&= \varepsilon^R && \text{by definition of reversal} \\
&= \varepsilon && \text{by definition of reversal} \\
&= w
\end{aligned}
$$

Case 2: Suppose that $w = ax$ for some symbol a and string x

Then,

$$
\begin{aligned}
\left(w^R\right)^R &= \left((ax)^R\right)^R \\
&= \left(x^R \bullet a\right)^R && \text{by definition of reversal} \\
&= a^R \bullet \left(x^R\right)^R && \text{by Problem 2} \\
&= a^R \bullet x && \text{by the inductive hypothesis} \\
&= a \bullet x && \text{by definition of reversal } (a = a\varepsilon; a^R = \varepsilon^R \bullet a = \varepsilon \bullet a = a) \\
&= ax && \text{by definition of concatenation} \\
&= w
\end{aligned}
$$

B.2 Lab 1B: Regular Expressions

A *regular expression* is compact notation for a language (that is, a set of strings). Formally, a regular language is one of the following:

- The symbol $\varnothing$ (representing the empty set)
- Any string (representing the set containing only that string)
- $\boldsymbol{R} + \boldsymbol{S}$ for some regular expressions R and S (representing alternation / union)
- $\boldsymbol{R} \cdot \boldsymbol{S}$ or $\boldsymbol{RS}$ for some regular expressions R and S (representing concatenation)
- $\boldsymbol{R}^*$ for some regular expression R (representing Kleene closure / unbounded repetition)

In the absence of parentheses, Kleene closure has highest precedence, followed by concatenation. For example:

$$\texttt{0} + \texttt{RE}^* = \texttt{0} + \texttt{R}(\texttt{E}^*) = \{\texttt{0}, \texttt{R}, \texttt{RE}, \texttt{REE}, \texttt{REEE}, \texttt{REEEE}, \texttt{REEEEE}, \texttt{REEEEEE}, \texttt{REEEEEEE}, \ldots\}$$

$$(\texttt{0} + \texttt{RE})^* = \{\varepsilon, \texttt{0}, \texttt{RE}, \texttt{RERE}, \texttt{RERERE}, \texttt{RERERERE}, \texttt{RERERERERE}, \texttt{RERERERERERE}, \ldots\}$$

$$(\texttt{0} + \texttt{R})\texttt{E}^* = \{\texttt{0}, \texttt{R}, \texttt{0E}, \texttt{RE}, \texttt{0EE}, \texttt{REE}, \texttt{0EEE}, \texttt{REEE}, \texttt{0EEEE}, \texttt{REEEE}, \texttt{0EEEEE}, \ldots\}$$

$$(\texttt{0} + \texttt{RE})^* = \{\varepsilon, \texttt{0}, \texttt{00}, \texttt{RE}, \texttt{000}, \texttt{0RE}, \texttt{RE0}, \texttt{0000}, \texttt{00RE}, \texttt{0RE0}, \texttt{RE00}, \texttt{RERE}, \ldots\}$$

Give regular expressions for each of the following languages over the binary alphabet {0, 1}. (For extra practice, find multiple regular expressions for each language.)

0. All strings.

$$\boxed{(\texttt{0} + \texttt{1})^*}$$

1. All strings containing the substring 000.

```
((0 + 1)* 000 (0 + 1)*)
```

2. All strings *not* containing the substring 000.

$$\boxed{(\texttt{1} + \texttt{01} + \texttt{001})^*(\varepsilon + \texttt{0} + \texttt{00})}$$

3. All strings in which every run of 0s has length at least 3.
4. All strings in which the *last* 1 appears before the *first* substring 000.
5. All strings containing at least three 0s.
6. Every string except 000. *[Hint: Don't try to be clever.]*

Appendix C: Problem Sets

C.1 Problem Set 1

C.1.1 Problem 1